Self-Supervised Keyframe Discovery for Horizon-Invariant Behavior Cloning

Prabin Kumar Rath
Arizona State University
prath4@asu.edu

Omkar Patil
Arizona State University
opatil3@asu.edu

Nakul Gopalan
Arizona State University
ng@asu.edu

Abstract

Behavior cloning (BC) in non-Markovian environments is a challenging problem because policies have to reason over contextual information over long horizons. Existing policy architectures rely on recurrent or attention-based mechanisms to capture long-term dependencies. However, recurrent models suffer from hidden-state collapse and gradient instability under backpropagation through time while attention-based models are fundamentally limited by context length. To address these issues, we propose Keyframe Mnemonics, a novel self-supervised method that *discovers* a set of information-critical observations (*mnemonics*) by learning an objective from randomly sampled past observations and using it as a reward for keyframe selection. We then train a BC policy that conditions on the discovered keyframes to model the action distribution. Under the simplicity bias assumption [1, 2], our formulation provides context retention guarantees over an infinite horizon, while maintaining a small set of decision-relevant keyframes in the policy's working memory. We evaluate our method on synthetic memory domains, where mnemonic-conditioned BC policies achieve 100% success rates (SR) and generalize to horizons orders of magnitude beyond training without performance degradation. Moreover, we evaluate on a memory-intensive robot manipulation benchmark, achieving a 13.9% average absolute SR improvement over the strongest baseline across 23 tasks. Finally, we demonstrate generalization with 80% SR at $20\times$ longer time horizons on a real robot. Code and videos available at `https://keyframe-mnemonics.github.io`.

1 Introduction

The Markov assumption for decision processes is that the agent's current state contains sufficient information to determine the optimal action with respect to its goal or objective [3]. In practice, however, real-world environments are non-Markovian and partially observable, where decision-making often requires retaining ("remembering") information over time to provide context for future actions [4, 5]. Recent advancements in policy learning have increasingly relied on behavior cloning (BC) [6, 7, 8], in which policies are highly dependent on short observation contexts with a working memory spanning only a few seconds into the past. This makes such BC policies brittle on tasks that require remembering information cues over long horizons [9]. Recurrent models have been the architecture of choice for behavior cloning in such memory intensive partially observable environments and are typically trained using Back-Propagation Through Time (BPTT) [10, 11]. However, BPTT is difficult to scale, as it requires sequential gradient accumulation over episode timesteps [12]. Truncated-BPTT tries to mitigate this issue but trades off information from distant past observations [13]. Moreover, recurrent models suffer from hidden-state collapse due to repeated representation updates over time [14, 12]. Attention-based architectures such as transformers [15] address many representation learning challenges, but their finite context length limits retention of the full history in long horizon domains. Furthermore, increasing context through frame stacking incurs

40th Conference on Neural Information Processing Systems (NeurIPS 2026).

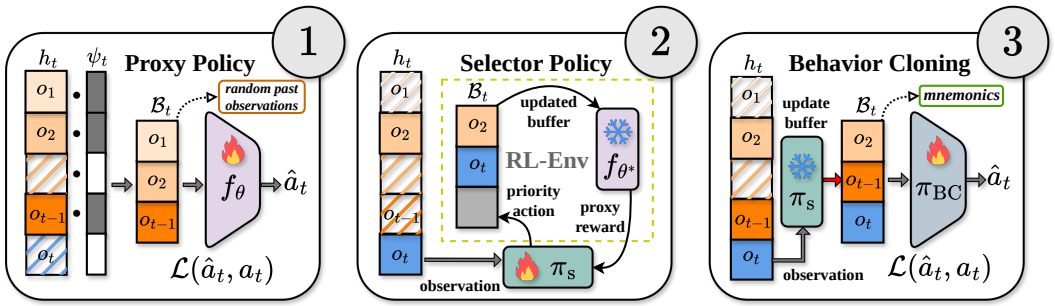


Figure 1: **Keyframe Mnemonics three-stage training for horizon-invariant behavior cloning. (1) Proxy policy training.** A random binary mask Ψ_k is applied to the observation history h_t to assemble a memory buffer $\mathcal{B}_t$ of size at most k, on which the proxy $f_\theta : \mathcal{B} \to \mathcal{A}$ is trained to predict the expert action a_t under a negative log-likelihood loss $\mathcal{L}$. Combinatorial support of training data (Lemma 1) biases f_θ toward keyframe-based hypotheses (Theorem 1, Proposition 1), so the frozen objective $J_{\theta^*}(\mathcal{B}_t) = \mathcal{L}(f_{\theta^*}(\mathcal{B}_t), a_t)$ scores keyframe-rich buffers favourably (Corollary 1; Section 2.1). **(2) Selector policy training.** The selector π_s is trained using PPO to populate $\mathcal{B}_t$ with keyframes using $J_{\theta^*}(\mathcal{B}_t)$ as reward function (Section 2.2). **(3) Behavior cloning.** The trained selector is used to construct a keyframe augmented dataset $\mathcal{D}^\dagger$, which is then used to train the BC policy $\pi_{\text{BC}}(a_t \mid o_t, \mathcal{B}_t)$ (Section 2.3). Throughout the figure, blue tiles denote the current observation o_t, orange tiles denote past frames in h_t, hatched tiles are frames not selected into $\mathcal{B}_t$.

quadratic compute complexity [15], degrades at extreme horizons [16], and is prone to overfitting spurious correlations with visual observations [17]. Many hybrid architectures have been proposed that improve information retention through dedicated memory mechanisms [16, 18, 19, 20, 21, 22, 23] or by learning recurrent summaries [24, 25, 26]. However, these methods remain susceptible to state collapse beyond the horizons encountered during training, often leading to severe performance degradation on tasks that require generalization to longer horizons.

To address these challenges in behavior cloning, we propose **Keyframe Mnemonics (KM)**, a policy learning method that decomposes the original non-Markovian decision-making problem into two subproblems: (i) discovering observations (*keyframes*) that contain information relevant to the action distribution, and (ii) training a BC policy on demonstrations augmented with the identified keyframes. Our keyframe discovery process is self-supervised and leverages the intrinsic simplicity bias of neural networks [1, 2, 27] to train a reward function that assigns high rewards to information-rich keyframes in the policy's context buffer. The reward learning process is driven by the mutual information advantage of keyframes with the expert action distribution, which is amplified through a novel training method that uses randomly sampled past observations as training samples for the reward function. We then train a selector policy using reinforcement learning to optimize the learned reward function and identify a set of decision-relevant keyframes, which we call *Mnemonics* ("something that assists memory"). Finally, we train BC policies conditioned on these mnemonics to match the expert action distribution. The trained policy is deployed together with the selector, which identifies keyframes at runtime and provides context to disambiguate actions under partial observability. For environments with sparse keyframes, this architectural decomposition enables *horizon-invariance* i.e., memory retention beyond the training horizon, allowing BC policies to generalize on longer time scales where existing hybrid memory methods fail. We extensively evaluate Keyframe Mnemonics on synthetic memory benchmarks (Add [28], Scattered-Copy [24], T-Maze [29]), grid worlds (MiniGrid Memory [30], LTMB-Hallway [31]), memory-intensive robot manipulation benchmark MIKASA-Robo [9], and additionally on a real robot, demonstrating behavior generalization without performance degradation far beyond the training horizon. Our contributions are:

- We propose a novel self-supervised algorithm to discover decision-relevant keyframes in non-Markovian expert demonstrations without explicit keyframe annotations (Figure 1, Algorithm 1).
- We provide a theoretical intuition of our method for environments with sparse keyframes, derive information retention guarantees, and support our hypotheses with empirical evidence.
- We demonstrate horizon-invariant performance of mnemonic-conditioned BC policies, with MLP and CNN policies achieving 100% SR in synthetic and grid memory domains, and Transformer

policies improving absolute SR by 13.9% on the MIKASA-Robo benchmark. Additionally, we show a flow-matching policy retaining 80% SR at $20\times$ longer test timesteps on a real robot.

2 Methodology

We assume a partially observable Markov decision process (POMDP) [4] $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \Omega, \rho_0)$, where $\rho_0 \in \Delta(\mathcal{S})$ denotes the initial state distribution. At timestep $t \in \{1, \dots, T\}$ with horizon $T \in \mathbb{N}^+$, the unobserved state is denoted by $s_t \in \mathcal{S}$, the agent receives an observation $o_t \in \mathcal{O}$, and takes an action $a_t \in \mathcal{A}$ based on a belief over the state, $b_t = P(s_t \mid h_t)$, where $h_t = \{o_1, \dots, o_t\}$ denotes the history of observations up to time t. Here, $\mathcal{T}(s_{t+1} \mid s_t, a_t)$ and $\Omega(o_{t+1} \mid s_{t+1}, a_t)$ denote the transition dynamics and observation function respectively. We omit the reward function $\mathcal{R}$ and discount factor γ from the notation, as they are not available in standard behavior cloning. Additionally, we assume access to an expert demonstration dataset $\mathcal{D} = \{\tau_i\}_{i=1}^{N}$, where each trajectory is given by $\tau_i = \{(o_t, a_t)\}_{t=1}^{T_i}$. We make the following assumptions over the dataset (Assumptions 1 and 2) and training dynamics for gradient descent (Assumptions 3 and 4), where $I(\cdot;\cdot)$ and $H(\cdot)$ denote mutual information and entropy, respectively.

Assumption 1 (Approximate belief sufficiency). The trajectory observations admit a partition into two disjoint subsets: a sparse subset $\mathcal{X}_{\text{key}}$ of size $C \ll T$ (*keyframes*) and the remaining $T - C$ observations $\mathcal{X}_{\text{bg}}$ (*background frames*). The keyframes $\mathcal{X}_{\text{key},t} \subset h_t$ up to time t contain observations with invariant task information and causal cues that remain consistent across the demonstration dataset, thereby forming an approximate sufficient statistic for the belief state b_t. In contrast, the background subset $\mathcal{X}_{\text{bg},t} \subset h_t$ contributes minimal information toward the belief state. This assumption imposes a sparse keyframe structure on observations in the demonstration dataset, which we empirically observe in several memory intensive partially observable domains evaluated in Section 3.

Assumption 2 (Action disambiguation by keyframes). Any ambiguity in the action distribution is primarily captured by the keyframes $\mathcal{X}_{\text{key}}$, with the following relationship in the mutual information $I(a_t; \mathcal{X}_{\text{key},t}) > I(a_t; \mathcal{X}_{\text{bg},t})$. That is, keyframes strictly provide more information for disambiguating the action than the background frames. Section 3 provides empirical evidence consistent with this assumption in the evaluated memory-intensive POMDPs.

Assumption 3 (Realizability of conditional programs by interpolating hypotheses). We combine the standard interpretation of conditional Kolmogorov complexity as the shortest program mapping inputs to outputs [32, 33] with the finite-sample expressivity of overparameterized neural networks, which can interpolate or memorize arbitrary training labels [34, 27, 35]. (Details in Appendix A).

Assumption 4 (Simplicity-biased SGD approximation). We assume SGD is biased towards selecting Kolmogorov simple functions, where low-complexity functions receive larger prior mass [1, 2, 36]. (Details in Appendix A).

We formally define *Mnemonics* as the set of keyframes up to time t, denoted by $\mathcal{X}_{\text{key},t} \subset h_t$, which provide sufficient information for a behavior cloning policy to generate actions unambiguously. Our goal is to learn a selector policy π_{s} that identifies keyframes from a continuous stream of observations and adds them to a memory buffer $\mathcal{B}_t$, maximizing its overlap with $\mathcal{X}_{\text{key},t}$. These mnemonics are then used to condition a behavior cloning policy π_{BC} that models the expert action distribution.

2.1 Leveraging Simplicity-bias for Keyframe Discovery

We formulate keyframe *discovery* as a self-supervised buffer optimization problem. Specifically, we define an objective $J(\cdot, f)$ over a proxy policy f mapping observation frames in the buffer, which collectively represent the belief state, to expert actions, $f : \mathcal{B} \to \mathcal{A}$, and optimize the buffer $\mathcal{B}$ subject to a memory budget k much smaller than the observation history ($k \ll T$). For $J(\cdot, f)$ to serve as an effective objective, it must provide a stronger learning signal (i.e., lower objective value) when $\mathcal{B}$ contains a higher proportion of keyframes than background frames. In this section, we introduce a training strategy for the proxy policy f that increases the complexity of the input–output mapping, thereby exploiting the simplicity bias of deep neural networks to encourage learning a hypothesis that favors keyframes over background frames due to their greater informational content. The resulting biased model can then be used to learn a selector π_{s} for keyframe discovery.

Algorithm 1 Masked-sampling based training of the proxy policy f_θ

Require: dataset $\mathcal{D}$, training memory budget k, proxy policy f_θ

for each training step **do**
  Sample $\tau \sim \mathcal{D}$ and timestep $t \in \{1, \dots, |\tau|\}$ ▷ uniform sampling of timestep
  $n \leftarrow \min\big(\text{Binomial}(t,\, 0.5),\; k\big)$ ▷ buffer occupancy under budget k
  Draw $\mathcal{I} \subset \{1, \dots, t\}$ with $|\mathcal{I}| = n$ ▷ uniform sampling without replacement
  $\Psi_k(i) \leftarrow \mathbf{1}[\, i \in \mathcal{I}\,]$ for $i = 1, \dots, t$ ▷ induced binary mask on h_t
  $\mathcal{B}_t \leftarrow \{\, o_i \in h_t \mid \Psi_k(i) = 1 \,\}$ ordered by i ▷ $|\mathcal{B}_t| \le k$
  $\hat{a}_t \leftarrow f_\theta(\mathcal{B}_t)$ ▷ predict expert action
  $\theta \leftarrow \theta - \eta\, \nabla_\theta\, \mathcal{L}(\hat{a}_t,\, a_t)$ ▷ update proxy using NLL loss
end for

Proxy Policy Training: To train the proxy policy f_θ, a neural network parameterized by θ, we construct input buffers using a random binary mask $\Psi_k \in \{0,1\}^{|h_t|}$ that independently selects past observations from the history h_t, with cardinality bounded by the memory budget k. We call this *masked-sampling* past observations that constructs the training-time memory buffer $\mathcal{B}_t = \{o_i \in h_t \mid \Psi_k(i) = 1\}$, which serves as input to the proxy policy $f_\theta : \mathcal{B} \to \mathcal{A}$ with $|\mathcal{B}_t| \le k$, where $\mathcal{B}$ denotes the space of all possible memory buffers. Algorithm 1 shows the masked-sampling method for training the proxy policy f_θ. Treating $(\mathcal{B}_t, a_t)$ as i.i.d. samples, we now analyze the hypothesis space $\mathcal{H}$ explored by f_θ during training through the lens of Kolmogorov complexity [33], denoted $K(f)$ for $f \in \mathcal{H}$ [1]. Let $\mathcal{H}^* \subset \mathcal{H}$ denote the set of interpolating hypotheses achieving zero empirical risk on the training dataset induced by masked-sampling. Assume $\mathcal{H}^*$ partitions into an invariant class $\mathcal{H}_{\text{in}}$, corresponding to hypotheses conditioned only on keyframes, and a non-invariant class $\mathcal{H}_{\neg\text{in}}$, which depend on background frames. Let $f_{\text{key}} \in \mathcal{H}_{\text{in}}$ denote keyframe-based and $f_{\text{bg}} \in \mathcal{H}_{\neg\text{in}}$ denote background-based hypotheses defined by mappings $f_{\text{key}} : \mathcal{X}_{\text{key}} \to \mathcal{A}$ and $f_{\text{bg}} : \mathcal{X}_{\text{bg}} \to \mathcal{A}$, where the inputs are constrained to at most k frames, consistent with f_θ.

Overview: First, we show that masked-sampling of past observations during training of f_θ induces combinatorially large support of training pairs (Lemma 1), which artificially increases the complexity of background-based hypotheses f_{bg} and effectively restricts the feasible hypothesis space (Theorem 1). Assuming that deep neural networks exhibit an implicit bias toward simpler hypotheses, we show that keyframe-based hypotheses f_{key} are learned with high probability (Proposition 1). Finally, we demonstrate that the prediction error of the trained proxy policy provides an optimizable signal to identify keyframe observations (Corollary 1).

Lemma 1 (Combinatorial cardinality of the sample space). *The total cardinality M of the space of distinct timestep-indexed training pairs $(\mathcal{B}_t, a_t)$ is given by:*

$$M = \sum_{i=1}^{N} \left(T_i + \sum_{j=1}^{k} \binom{T_i + 1}{j + 1} \right). \tag{1}$$

Masked-sampling induces a combinatorially large finite support of training pairs, whose cardinality M admits a closed-form expression via the Hockey-stick identity [38]. (Proof in Appendix A). □

Theorem 1 (Expected gap in hypothesis complexity). *Under the assumption that training pairs are sampled i.i.d. from a sufficiently large support of size M (Lemma 1), the expected Kolmogorov complexity of the minimum-complexity background-based hypothesis f^*_{bg} is strictly greater than that of the keyframe-based hypothesis f^*_{key}.*

$$\mathbb{E}[K(f^*_{\text{bg}})] > \mathbb{E}[K(f^*_{\text{key}})]. \tag{2}$$

Using results from [32, 33], we show that, under the stated assumptions, the complexity gap between background-based and keyframe-based hypotheses grows linearly with the masking-induced support size M according to $M\big(I(A; \mathcal{X}_{\text{key}}) - I(A; \mathcal{X}_{\text{bg}})\big)$. Thus, any information advantage of keyframes is amplified by the large support size M, rendering background-based hypotheses intractable for the proxy policy f_θ. (Proof in Appendix A). □

[1] For notational simplicity, we use the same variable f to denote both the proxy policy and the hypothesis learned by the proxy policy, and use Kolmogorov complexity as a measure of hypothesis description length [37].

Proposition 1 (Conditional simplicity-bias preference). *Under the simplicity-bias assumption and the complexity gap established in Theorem 1, Stochastic Gradient Descent (SGD) is biased toward the invariant hypothesis class $\mathcal{H}_{\text{in}}$. Given a masking-induced sample distribution with support size M (Lemma 1), the probability of selecting an invariant hypothesis is approximated by*

$$P_{\text{SGD}}(\mathcal{H}_{\text{in}} \mid \mathcal{D}) \approx \frac{1}{1 + 2^{-\left[M\left(I(A;\mathcal{X}_{\text{key}}) - I(A;\mathcal{X}_{\text{bg}})\right) - \mathcal{O}(1)\right]}}. \tag{3}$$

Neural networks and SGD are biased toward Kolmogorov-simple functions [1, 2, 36]. Hence, the positive hypothesis complexity gap from Theorem 1 drives $P_{\text{SGD}}(\mathcal{H}_{\text{in}} \mid \mathcal{D})$ close to 1, biasing f_θ toward learning an invariant keyframe-based hypothesis. (Proof in Appendix A). □

Corollary 1 (Keyframe recovery from the frozen loss objective). *Let us define a subset-selection objective function*

$$J_{\theta^*}(\mathcal{B}) = \mathbb{E}[\mathcal{L}(f_{\theta^*}(\mathcal{B}), A)] \tag{4}$$

over a frozen predictor f_{θ^*} obtained under Proposition 1 after training with negative log-likelihood loss $\mathcal{L}$, where f_{θ^*} represents a near-optimal invariant hypothesis in $\mathcal{H}_{\text{in}}$. Consider candidate buffer subsets $\mathcal{C}_k = \{\mathcal{B}_t \subseteq h_t : |\mathcal{B}_t| \leq k\}$, and let $\mathcal{B}_{\text{opt}} \in \arg\min_{\mathcal{B}\in\mathcal{C}_k} J_{\theta^*}(\mathcal{B})$. If keyframe subsets $\mathcal{B} \subset \mathcal{X}_{\text{key}}$ have a positive action-information margin over all non-keyframe candidate subsets, and this margin dominates the approximation error of the frozen predictor f_{θ^*}, then the loss objective $J_{\theta^*}(\mathcal{B})$ recovers the keyframes up to information-equivalent subsets. In particular, if $\mathcal{X}_{\text{key}}$ is unique under the memory budget k, then $\mathcal{B}_{\text{opt}} = \mathcal{X}_{\text{key}}$.

Optimizing $J_{\theta^*}(\mathcal{B})$ is equivalent to minimizing the information gap $I(A;\mathcal{X}_{\text{key}}) - I(A;\mathcal{B})$ over $\mathcal{C}_k$. Under Assumption 2, this drives $\mathcal{B}$ toward $\mathcal{X}_{\text{key}}$. (Proof in Appendix A). □

The insight of our approach is that by leveraging the natural simplicity bias of neural networks, we can automatically discover informative keyframes by partitioning a dataset based on the mutual information gap between observations and the action distribution. Our method is analogous to masked data augmentation [39, 40] but along the temporal axis, where our masking strategy increases the difficulty of learning spurious correlations and biases the model toward invariant temporal mappings. Theorem 1 provides theoretical intuition for this phenomenon; see Section 3.1, where we provide empirical evidence across synthetic benchmark domains.

2.2 Selector Optimization for Keyframe Identification

Given the learned objective function, we train a selector policy π_{s} to identify keyframes. The selector serves two purposes: (i) extract keyframes from the demonstration dataset for constructing mnemonics that can be used to train a downstream policy π_{BC}, and (ii) identify keyframes from a continuous stream of observations during inference to populate the memory buffer for π_{BC}. However, note that the objective $J_{\theta^*}(\mathcal{B})$ is defined over buffer inputs rather than individual observations. Furthermore, the number of possible buffer states is combinatorially large (Lemma 1), making exhaustive evaluation non-trivial. To address this, we formulate keyframe identification as a deterministic Markov Decision Process (MDP) over the demonstration dataset, modeling the buffer $\mathcal{B}$ as a priority queue and using a selector policy π_{s} to assign priority scores that update the buffer. This reformulates the problem from optimizing over buffer states to learning a selector policy that prioritizes keyframe observations.

The selection MDP is defined by the tuple $\langle \mathcal{O}, \mathcal{P}, \mathcal{K}, R, \gamma \rangle$ over offline demonstration trajectories $\tau_i \in \mathcal{D}$, where $\tau_i = \{(o_t, a_t)\}_{t=1}^{T_i}$. At each timestep $t \in \{1, \ldots, T_i\}$, the selector policy $\pi_{\text{s}}(p_t \mid o_t)$ receives an observation $o_t \in \mathcal{O}$ and outputs a priority action $p_t \in \mathcal{P}$, which is used to update the buffer $\mathcal{B}$, modeled as a priority queue with memory budget k. The MDP tuple is specified as follows. **State ($\mathcal{O}$)** is the current observation o_t from a trajectory $\tau_i \in \mathcal{D}$. An **Action ($\mathcal{P}$)** is a priority score $p_t \in [0, 1]$. **Transition ($\mathcal{K}$)**: The transition is deterministic, with the next state given by the subsequent observation o_{t+1} in τ_i. The buffer is updated as $\mathcal{B}_t = \mathcal{B}_{t-1}(o_t, p_t)$ according to an eviction rule that determines whether o_t is inserted into or rejected from the buffer. We use a **Discount Factor (γ)** $= 0.99$. The **Reward (R)** function depends on the updated buffer $\mathcal{B}_t$ and a normalization constant $\lambda \in (0, 1)$ (typically a value close to 0).

$$R(\mathcal{B}_t) = \underbrace{T_i \cdot \frac{e^{J^{j=t}_{\theta^*}/\kappa}}{\sum_{j=1}^{T_i} e^{J^{j}_{\theta^*}/\kappa}}}_{\text{surprise}} \cdot \underbrace{\frac{\log\left(J_{\theta^*}(\mathcal{B}_t) + \lambda\right)}{\log\left(\lambda\right)}}_{\text{reward}}. \tag{5}$$

We use a normalized logarithmic reward which increases sharply toward 1 as the objective value approaches 0, providing a strong optimization signal even for small changes in the memory buffer. This formulation encourages π_{s} to assign higher priorities to observations that minimize the learned objective. The reward is further scaled by a surprise [24, 41] coefficient weighted by episode length, which captures action uncertainty. Intuitively, this assigns higher reward weight to timesteps with larger objective values, which corresponds to higher action entropy under the proxy policy. Such timesteps are typically decision-critical, where actions depend strongly on the memory buffer. Hence, emphasizing them focuses optimization on resolving keyframes where they matter most.

The surprise coefficients are precomputed for each trajectory using a temperature (κ)-scaled softmax over the expected objective values across timesteps. The expected objective values are computed over a few epochs of masked-sampled input buffers by evaluating the objective $J_{\theta^*}(\mathcal{B})$ at each timestep in the demonstration dataset $\mathcal{D}$. We optimize π_{s} using Proximal Policy Optimization (PPO) [42], which enables stable training while promoting exploration through entropy regularization. The memory buffer employs different priority strategies during training and inference. During training, a FIFO queue penalizes the selector policy for assigning low priority to observations that have long-term influence on actions, whereas during inference, a LIFO queue preserves high-priority observations, ensuring their retention over long horizons. See Appendix B.1 for algorithms on priority queue eviction rules used during training and inference.

2.3 Training BC Policy with Keyframe Mnemonics

A trained selector policy is used to generate mnemonics, represented by the buffer state $\mathcal{B}_t$ with $|\mathcal{B}_t| \leq k$, which approximates the keyframes up to time t, i.e., $\mathcal{X}_{\mathrm{key},t} \subset h_t$. We now present a conditional guarantee for context retention in the memory buffer assuming an optimal selector, which enables mnemonics-conditioned BC policies to generalize to an infinite horizon (Theorem 2).

Theorem 2 (Infinite-horizon keyframe retention). *Assume a POMDP in which the expert behavior strictly satisfies Assumptions 1 and 2. Let π_{s}^* be an optimal selector minimizing the frozen objective $J_{\theta^*}(\mathcal{B}_t)$ (Corollary 1). Suppose a unique keyframe set $\mathcal{X}_{\mathrm{key}}$ exists that has the highest mutual information with the action and fits within the memory budget k. Then, as $T \to \infty$, π_{s}^* assigns higher priority to keyframes than to background frames, ensuring that the priority queue retains the keyframe set for the entire horizon T without eviction, i.e., $\mathcal{B}_T = \mathcal{X}_{\mathrm{key}}$.*

An optimal selector minimizes the learned objective $J_{\theta^*}(\mathcal{B}_t)$ at every time step t, which corresponds to recovering the keyframe set $\mathcal{X}_{\mathrm{key},t}$ at each t. (Proof in Appendix A). □

Ideally, the memory buffer should retain the full context needed to disambiguate actions at each timestep t. However, in complex domains such as robot manipulation, our assumptions may not hold strongly, and actions may have some dependency on the background observation set. To address this, we additionally include the current observation o_t and construct an augmented dataset $\mathcal{D}^\dagger$ with trajectories $\tau_i^\dagger = \{(\mathcal{B}_t, o_t, a_t)\}_{t=1}^{T_i}$, which is used to train the BC policy $\pi_{\mathrm{BC}}(a_t \mid o_t, \mathcal{B}_t)$. See Appendix C and D for details on policy training.

3 Experiments and Result Analysis

In this section, we present experiments validating the effectiveness of the Keyframe Mnemonics (KM) method. We first provide empirical evidence supporting our theoretical analysis of keyframe discovery from Section 2.1. We then evaluate mnemonic-conditioned BC policies along two central questions: **(1)** How effectively does the selector identify keyframes across diverse memory benchmarks, ranging from synthetic domains to robot manipulation tasks? **(2)** Can mnemonic-conditioned BC policies generalize beyond the training horizon and exhibit horizon-invariant behavior cloning?

3.1 Synthetic Domain Benchmarks and Convergence Analysis

Domains. We evaluate on three synthetic memory sequence-to-sequence tasks and two grid-based environments. These domains satisfy Assumptions 1 and 2, making them suitable for empirical analysis of our method under the stated assumptions. **Add** [28] requires the policy to emit the sum of two marked floating point numbers from a sequence. **Scattered Copy** [24] interleaves a small set of n relevant tokens with noise tokens, and the policy must recover the relevant tokens in the same order

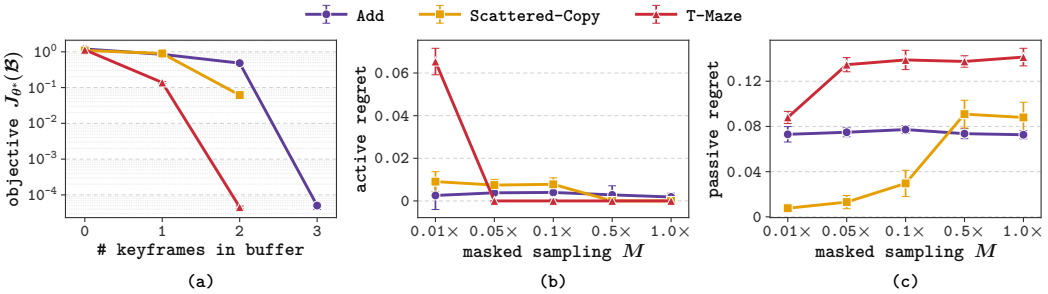


Figure 2: **Empirical evidence for Keyframe Mnemonics.** (a) Adding *oracle* keyframes to the context buffer of the proxy policy leads to decrease in objective value across three problem domains (Corollary 1). (b) Active memory regret remains high for small masked-sampling support sizes M and converges to zero as the support size increases (Theorem 1). (c) Passive memory regret grows with M as the proxy policy fits the keyframe-based hypothesis. Note: Here, $M = 1.0\times$ corresponds to a calibrated value, chosen empirically (Appendix C.4), that is sufficient according to Theorem 1 and yields mnemonics for horizon-invariant BC policies, as shown in Figure 3.

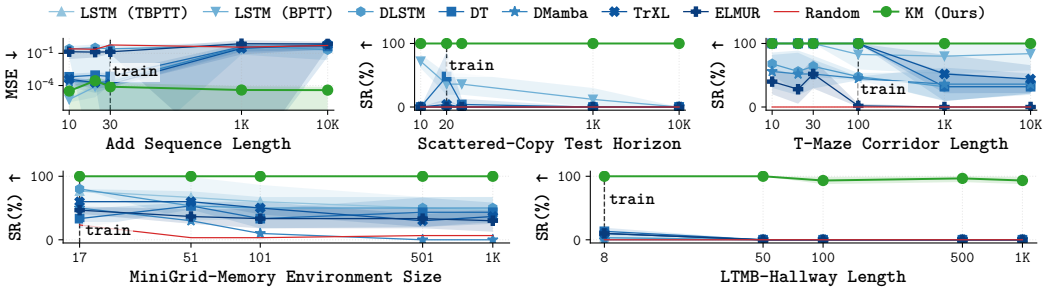


Figure 3: **Horizon-invariance in synthetic and grid domains.** Top row: Add, Scattered-Copy, T-Maze. Bottom row: MiniGrid-Memory, LTMB-Hallway. Each curve shows mean $\pm$ std over 5 seeds. Band width is one std-dev. Vertical dashed line marks the maximum training horizon. Lower is better for MSE; higher is better for Success Rate. KM maintains near-perfect performance across orders of magnitude of horizon scaling. Most baselines degrade outside the training range.

as observed. In **Active T-Maze** [29] the agent must remember an oracle observation from the start of a long corridor to make a binary decision at the final junction. **MiniGrid-Memory** [30] is a similar memory task with a more diverse grid-based observation space. **LTMB-Hallway** [31] is a visual memory benchmark where the agent must match a key observation seen at the episode start to one of several targets uniformly placed along a long hallway. The policy must compare each observed door with the remembered visual cue and select the matching one. See Appendix C.1 for details on ground truth keyframes in each domain.

Proxy policy objective. Figure 2(a) shows the learned objective value $J_{\theta^*}(\mathcal{B})$ as a function of the number of keyframes in the input buffer. Across three synthetic tasks, we observe a decrease in the objective value as more keyframes are added to the buffer, supporting Corollary 1.

Memory regret metrics. Let $\mathcal{L}_t^\pi$ denote the per-timestep proxy policy loss when the buffer is populated by a selection policy π. We compare the trained selector π_{s} against an *oracle* selector $\pi^\star$ that assigns priority 1 to keyframes and 0 otherwise, and a *passive* baseline π_{rnd} that samples priorities from $\mathcal{U}[0,1]$. For memory retention analysis, we define active memory regret as $r_{\mathrm{active}} = \mathbb{E}\left[\frac{1}{T}\sum_{t=1}^{T}(\mathcal{L}_t^{\pi_{\mathrm{s}}} - \mathcal{L}_t^{\pi^\star})\right]$ and passive memory regret as $r_{\mathrm{passive}} = \mathbb{E}\left[\frac{1}{T}\sum_{t=1}^{T}(\mathcal{L}_t^{\pi_{\mathrm{rnd}}} - \mathcal{L}_t^{\pi^\star})\right]$. r_{passive} characterizes the penalty incurred by not actively selecting what to retain, while r_{active} measures how close the trained selector is to the oracle. The desired regime for long horizon information retention is $r_{\mathrm{passive}} \gg r_{\mathrm{active}} \approx 0$. We vary the masked-sampling support size M across three synthetic domains and report the memory regret incurred using the learned objective in Figure 2(b,c). With sufficiently large calibrated support, the active regret drops to zero, while passive regret gradually increases with M, providing empirical evidence for Theorem 1.

Baselines and horizon-invariance experiment. We compare KM (mnemonic-conditioned MLP/Transformer policies) against seven competitive sequence models, all trained on BC datasets with favorable parameter budgets. The set comprises LSTM (BPTT/TBPTT) [28], an LSTM trained with backpropagation through time, Decision-LSTM (DLSTM) [43], Decision Transformer (DT) [44], Decision-Mamba (DMamba) a DT-like state-space model [45], Transformer-XL (TrXL) with relative position encoding and a recurrent memory cache [46], ELMUR, a long-horizon mixture-of-experts memory transformer [16], and Random, a stochastic baseline that we include as a noise floor. For each task domain, we evaluate across a range of horizons that includes both in-distribution and out-of-distribution settings, spanning orders of magnitude beyond training distribution. Results in Figure 3 show that *KM maintains near-perfect performance across all horizons on every task, whereas all baseline policies collapse beyond the training range*. To probe the limits of generalization we also evaluated KM at two further horizons 10^5 and 10^6 time steps. KM retained *equivalent performance* to its in-distribution behavior on all three synthetic tasks; see Appendix E, Table 12.

3.2 Robot Manipulation Benchmarks and Real-world Experiment

Memory benchmark. We evaluate the performance of our method on memory-intensive robot manipulation using the `MIKASA-Robo` [9] benchmark which has 32 tabletop manipulation tasks that evaluate object, spatial, sequential, and capacity memory capabilities of visuomotor policies. In each task the policy observes critical information at moments when the cues are visible, and must then act on that information after the cues are no longer in the observation. For example, in `RememberColor3`, the agent is briefly shown a colored cube as a cue; after the cue is removed from the observation, the robot must reach to and touch the matching cube among the three differently-colored cube options, requiring the policy to retain the cue across the time gap between observation and choice action. `MIKASA-Robo` thus evaluates the policy's ability to develop a relevant context from past observations to inform its action at the current state. To further evaluate the memory-retention capabilities of policies, that is, how long the policy can hold the necessary information for decision-making, we benchmark the policies on `RememberColor3-`ΔT with cue-to-choice time delays ΔT larger than what was seen during training.

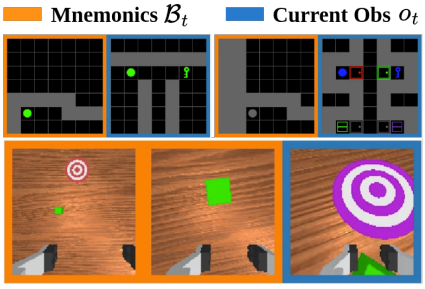


Figure 4: **Discovered mnemonics representing keyframes in grid and robot manipulation domains.**

Baselines. For memory benchmarking, we compare our method on all 32 tasks of the `MIKASA-Robo` against a broad set of baselines including sequence models (Decision Transformer [44]), offline RL (Conservative Q-Learning [47]), Behavior Cloning, Diffusion Policies [7] and the recent state-of-the-art method `ELMUR` [16]. Here, `RATE` [48] and `ELMUR` [16] serve as strong hybrid memory architecture baselines for comparison on tasks that demand memory capabilities from policies. For evaluation of memory retention capabilities, we compare our method with `ELMUR` and `PTP` [49]. `PTP` sub-samples frames from past observations for policy conditioning during training, in comparison to our learned approach of selecting relevant keyframes.

Experimental setup. We train Keyframe Mnemonics on the default dataset released by `MIKASA-Robo`. All results are reported as mean and standard error over 3 different seeds with 100 evaluations per seed. For the task designed to evaluate memory retention in manipulation tasks `RememberColor3-`ΔT, we train all baselines with the same dataset and a $\Delta T = 15$. During evaluation, we evaluate each baseline from $\Delta T = 15$ to $\Delta T = 150$ at intervals of 20 timesteps, and then at $\Delta T = 200, 250$. We refer the reader to the Appendix D for more details on the training configuration and the hyperparameters.

Results on `MIKASA-Robo`. Figure 5 summarizes the absolute success-rate improvement of Keyframe Mnemonics over the strongest published baseline, `ELMUR`, on the 23 `MIKASA-Robo` tasks. Keyframe Mnemonics outperforms `ELMUR` on 17 of the 23 memory-intensive visual robot manipulation tasks and improves average absolute SR by 13.9%; see full table of results in Appendix D. These results show that our self-supervised approach to keyframe identification yields policies that effectively leverage past observational cues from the environment to inform their actions.

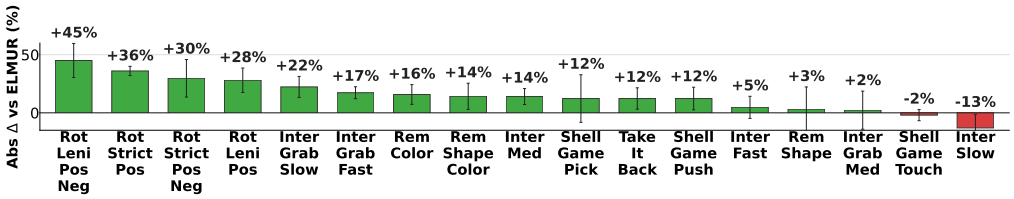
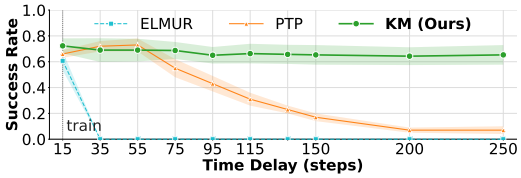
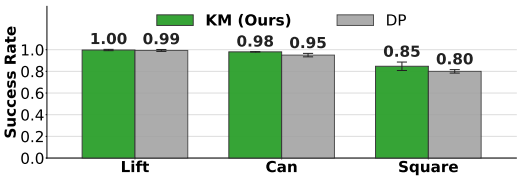


Figure 5: **Results across the 23 `MIKASA-Robo` tasks.** Keyframe Mnemonics improves on the strongest baseline (`ELMUR`) by an average of $+13.9$ absolute success rate, outperforming `ELMUR` on 17 of the 23 tasks. Results for tasks `RememberShape3/5/9`, `RememberColor3/5/9` and `RememberShapeColor` are aggregated, and the full table of results can be found in the Appendix D.



(a) Memory retention on `RememberColor3-`ΔT under cue-to-choice delays beyond the training horizon (marked at 15). `ELMUR` collapses immediately, `PTP` degrades gradually, while Keyframe Mnemonics sustains its training-time success rate up to 250 steps.



(b) Keyframe Mnemonics matches the diffusion-policy (DP) [7] baseline on three `Robomimic` tasks (`Lift`, `Can`, `Square`) where memory provides no benefit, confirming it does not regress in the no-memory regime.

Figure 6: **Horizon-invariance in robot manipulation domain and results on Markovian tasks.**

Horizon-invariance experiment. Figure 6a evaluates how long each policy can hold the relevant cue once it is no longer visible. All three methods are trained at $\Delta T = 15$, but their out-of-distribution behavior diverges sharply: `ELMUR` collapses to near-zero success by $\Delta T = 35$, `PTP` retains roughly half its training performance through $\Delta T = 75$ before degrading, while Keyframe Mnemonics sustains its training-time success rate of $\sim 70\%$ across the full evaluated range up to $\Delta T = 250$. This highlights the advantages of our learned approach to keyframe identification over sub-sampling of frames from the past, as performance of `PTP` deteriorates once the cue recedes from its observational context, while Keyframe Mnemonics keeps holding it in the policy's context.

No regression on `Robomimic` tasks. Figure 6b shows that Keyframe Mnemonics matches the performance of diffusion-policy baseline trained without keyframes on three standard `Robomimic` manipulation tasks (`Lift`, `Can`, `Square`), confirming that the keyframe-conditioning does not deteriorate performance on Markovian benchmarks.

Real-world robot experiment. We evaluate KM on the `RememberColor2-`ΔT task, where the robot observes a red or green cube for 3–5 s. The cue is then removed, and after a delay, two cubes appear 30 cm apart. The robot must reach for the previously observed color. We collect 30 VR-teleoperated demonstrations (15 per color) with $\sim$5 s delays and evaluate at delays up to $20\times$ longer timesteps. Results shown in Figure 7.

Table 1: Training time.

Method / Stage	Time
KM (Ours)	**10h 29m**
Proxy	1h 15m
Selector	5h 46m
$\mathcal{D}^{\dagger}$ gen.	1h 23m
BC policy	2h 05m
`PTP` [49]	**19h 34m**
`ELMUR` [16]	**1h 23m**

Compute and training time. Table 1 reports training time for `RememberColor3-`ΔT on a single workstation with an NVIDIA RTX 3090 GPU and 12th-Gen Intel Core i9 CPU, with KM decomposed by training stages. KM trains faster than `PTP`, which conditions on subsampled frame buffers, while `ELMUR` is the fastest to train. We also trained `ELMUR` for $\sim$10 hours to match KM's training budget, but it remained at 0% SR for $\Delta T > 15$ and failed to generalize beyond the training horizon even with further training.

4 Related Work

Explicit memory modeling. Behavior cloning policies with dedicated memory have been developed using embedding caching [16], learned memory compression models [24, 25, 19, 18, 22, 23], temporal



Figure 7: **Horizon-invariance in real-world** `RememberColor2-`ΔT **task.** From left to right: Expert user provides demonstrations with cue-to-choice delay of ~ 5 seconds. During deployment, KM policy generalizes to a cue-to-choice delay of ~ 100 seconds. Performance of `DP` [7] and `PTP` [49] deteriorates while KM retains average 80% SR at $20\times$ longer test timesteps. See the webpage `https://keyframe-mnemonics.github.io` for policy rollout videos.

state aggregation [20, 49], and hybrid combinations of these approaches [21, 26]. However, these methods generally construct compressed state representations that can blur information over long horizons. In contrast, our method preserves information-rich keyframes regardless of changes in horizon length and generalizes to timescales far beyond those seen during training.

BC with keyframe annotations. Prior work on keyframe-conditioned behavior cloning has relied on manual keyframe annotations [31, 50, 51, 52] or task-specific prompts for VLM-based selection [53]. However, such approaches require additional supervision derived from oracle reasoning for each task. In contrast, our keyframe identification method is self-supervised, grounded in statistical learning principles, and requires no task-level oracle annotations. Therefore, we do not compare against methods that require additional data beyond expert demonstrations for policy training.

Learning to manage policy context. RL has been used in prior works to learn keyframe selection policies for optimizing context buffers [54, 55]. However, these methods have only been studied in simple grid and MuJoCo control environments, whereas we demonstrate that Keyframe Mnemonics generalizes to complex robot manipulation tasks in both simulation and the real world.

Simplicity-bias in neural networks. Simplicity-bias is a well-studied property of neural networks, reflecting their tendency to prefer simpler patterns over more complex ones [27, 1, 56, 57, 2]. Recent work has leveraged simplicity bias to improve RL performance [58], but this area remains largely underexplored, particularly in how such biases can be effectively utilized. We present one of the first methods that explicitly leverage this bias for behavior cloning in POMDP domains.

5 Conclusion

We presented a novel method for enabling horizon-invariant behavior cloning in partially observable memory intensive environments. Our approach uses a training strategy that samples random past observations to train a reward model, which is then optimized with reinforcement learning to learn a selector policy that discovers decision-relevant keyframes in a fully self-supervised manner. We provided theoretical analysis and intuition for our algorithm, along with empirical evidence supporting the proposed theory. Furthermore, we evaluated standard behavior cloning policy architectures conditioned on learned mnemonics, achieving state-of-the-art performance across diverse memory-intensive benchmarks, ranging from synthetic tasks to complex robot manipulation environments. We hope our work provides a step toward more reliable long-horizon behavior cloning by enabling policies to retain compact, decision-relevant context over extended horizons.

Limitations of our approach. The memory budget k is a hyperparameter that is difficult to predetermine, which can lead to multiple information-equivalent buffer states and limit the number of correct keyframes retained in the buffer. In such cases, storing more keyframes in the inference-time priority queue can provide richer context for the policy. Each demonstration used in this work must contain at least one keyframe. This limits the current algorithm in multi-task or "play demonstration" settings, where keyframes may vary significantly across tasks or may not be present in every trajectory. Addressing *horizon-invariance* for high-capacity memory remains an open problem, with the potential to enable more capable non-Markovian behavior cloning methods.

Acknowledgments

This material is based upon work supported by the Air Force Office of Scientific Research under award number FA9550-24-1-0239. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the United States Air Force. We also want to thank our anonymous reviewers for providing great feedback and making the work stronger. Additionally, we thank Kunal Garg for providing computational resources for our simulation experiments, Prabhath Adireddi for helping with the robot experiments, Weiwei Gu for proofreading the theoretical results, and Yifan Zhou for the initial discussions.

References

[1] Guillermo Valle-Perez, Chico Q Camargo, and Ard A Louis. Deep learning generalizes because the parameter-function map is biased towards simple functions. *arXiv preprint arXiv:1805.08522*, 2018.

[2] Chris Mingard, Henry Rees, Guillermo Valle-Pérez, and Ard A Louis. Deep neural networks have an inbuilt occam's razor. *Nature Communications*, 16(1):220, 2025.

[3] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.

[4] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2):99–134, 1998.

[5] Mikko Lauri, David Hsu, and Joni Pajarinen. Partially observable markov decision processes in robotics: A survey. *IEEE Transactions on Robotics*, 39(1):21–40, 2022.

[6] Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[7] Cheng Chi, Siyuan Feng, Yilun Du, Zhenjia Xu, Eric Cousineau, Benjamin Burchfiel, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[8] Kento Kawaharazuka, Jihoon Oh, Jun Yamada, Ingmar Posner, and Yuke Zhu. Vision-language-action models for robotics: A review towards real-world applications. *IEEE Access*, 13: 162467–162504, 2025. doi: 10.1109/ACCESS.2025.3609980.

[9] Egor Cherepanov, Nikita Kachaev, Alexey K Kovalev, and Aleksandr I Panov. Memory, benchmark & robots: A benchmark for solving complex tasks with reinforcement learning. *arXiv preprint arXiv:2502.10550*, 2025.

[10] Paul J Werbos. Backpropagation through time: what it does and how to do it. *Proceedings of the IEEE*, 78(10):1550–1560, 2002.

[11] Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. Babyai: A platform to study the sample efficiency of grounded language learning. *arXiv preprint arXiv:1810.08272*, 2018.

[12] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318. Pmlr, 2013.

[13] Corentin Tallec and Yann Ollivier. Unbiasing truncated backpropagation through time. *arXiv preprint arXiv:1705.08209*, 2017.

[14] Sepp Hochreiter, Yoshua Bengio, Paolo Frasconi, Jürgen Schmidhuber, et al. Gradient flow in recurrent nets: the difficulty of learning long-term dependencies, 2001.

[15] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[16] Egor Cherepanov, Alexey K Kovalev, and Aleksandr I Panov. Elmur: External layer memory with update/rewrite for long-horizon rl. *arXiv preprint arXiv:2510.07151*, 2025.

[17] Pim De Haan, Dinesh Jayaraman, and Sergey Levine. Causal confusion in imitation learning. *Advances in neural information processing systems*, 32, 2019.

[18] Haoquan Fang, Markus Grotz, Wilbert Pumacay, Yi Ru Wang, Dieter Fox, Ranjay Krishna, and Jiafei Duan. Sam2act: Integrating visual foundation model with a memory architecture for robotic manipulation. *arXiv preprint arXiv:2501.18564*, 2025.

[19] Myungkyu Koo, Daewon Choi, Taeyoung Kim, Kyungmin Lee, Changyeon Kim, Younggyo Seo, and Jinwoo Shin. Hamlet: Switch your vision-language-action model into a history-aware policy. *arXiv preprint arXiv:2510.00695*, 2025.

[20] Huiwon Jang, Sihyun Yu, Heeseung Kwon, Hojin Jeon, Younggyo Seo, and Jinwoo Shin. Contextvla: Vision-language-action model with amortized multi-frame context. *arXiv preprint arXiv:2510.04246*, 2025.

[21] Hao Shi, Bin Xie, Yingfei Liu, Lin Sun, Fengrong Liu, Tiancai Wang, Erjin Zhou, Haoqiang Fan, Xiangyu Zhang, and Gao Huang. Memoryvla: Perceptual-cognitive memory in vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2508.19236*, 2025.

[22] Nhat Chung, Taisei Hanyu, Toan Nguyen, Huy Le, Frederick Bumgarner, Duy Minh Ho Nguyen, Khoa Vo, Kashu Yamazaki, Chase Rainwater, Tung Kieu, et al. Rethinking progression of memory state in robotic manipulation: An object-centric perspective. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 3407–3415, 2026.

[23] Liufan Tan, Jiale Li, and Gangshan Jing. Memoact: Atkinson-shiffrin-inspired memory-augmented visuomotor policy for robotic manipulation. *arXiv preprint arXiv:2603.18494*, 2026.

[24] Artyom Sorokin, Nazar Buzun, Leonid Pugachev, and Mikhail Burtsev. Explain my surprise: Learning efficient long-term memory by predicting uncertain outcomes. *Advances in Neural Information Processing Systems*, 35:36875–36888, 2022.

[25] Yulin Zhou, Yuankai Lin, Fanzhe Peng, Jiahui Chen, Kaiji Huang, Hua Yang, and Zhouping Yin. Mtil: Encoding full history with mamba for temporal imitation learning. *IEEE Robotics and Automation Letters*, 2025.

[26] Yuheng Lei, Zhixuan Liang, Hongyuan Zhang, and Ping Luo. Vpwem: Non-markovian visuomotor policy with working and episodic memory. *arXiv preprint arXiv:2603.04910*, 2026.

[27] Devansh Arpit, Stanisław Jastrzębski, Nicolas Ballas, David Krueger, Emmanuel Bengio, Maxinder S Kanwal, Tegan Maharaj, Asja Fischer, Aaron Courville, Yoshua Bengio, et al. A closer look at memorization in deep networks. In *International conference on machine learning*, pages 233–242. PMLR, 2017.

[28] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8): 1735–1780, 1997.

[29] Bram Bakker. Reinforcement learning with long short-term memory. *Advances in neural information processing systems*, 14, 2001.

[30] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo Perez-Vicente, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and J K Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *Advances in Neural Information Processing Systems*, 36:73383–73394, 2023.

[31] William Yue, Bo Liu, and Peter Stone. Learning memory mechanisms for decision making through demonstrations. *arXiv preprint arXiv:2411.07954*, 2024.

[32] Peter D Grünwald, Paul MB Vitányi, et al. Algorithmic information theory. *Handbook of the Philosophy of Information*, pages 281–320, 2008.

[33] Ming Li, Paul Vitányi, et al. *An introduction to Kolmogorov complexity and its applications*, volume 3. Springer, 2008.

[34] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning requires rethinking generalization. *arXiv preprint arXiv:1611.03530*, 2016.

[35] Chulhee Yun, Suvrit Sra, and Ali Jadbabaie. Small relu networks are powerful memorizers: a tight analysis of memorization capacity. *Advances in neural information processing systems*, 32, 2019.

[36] Chris Mingard, Guillermo Valle-Pérez, Joar Skalse, and Ard A Louis. Is sgd a bayesian sampler? well, almost. *Journal of Machine Learning Research*, 22(79):1–64, 2021.

[37] Peter D Grünwald. *The minimum description length principle*. MIT press, 2007.

[38] Blaise Pascal. *Traité du triangle arithmetique: avec quelques autres petits traitez sur la mesme matiere*. Chez Guillaume Desprez, 1978.

[39] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*, pages 4171–4186, 2019.

[40] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16000–16009, 2022.

[41] Chuan Wen, Jierui Lin, Jianing Qian, Yang Gao, and Dinesh Jayaraman. Keyframe-focused visual imitation learning. *arXiv preprint arXiv:2106.06452*, 2021.

[42] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[43] Max Siebenborn, Boris Belousov, Junning Huang, and Jan Peters. How crucial is transformer in decision transformer? *arXiv preprint arXiv:2211.14655*, 2022.

[44] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 34:15084–15097, 2021.

[45] Qi Lv, Xiang Deng, Gongwei Chen, Michael Y Wang, and Liqiang Nie. Decision mamba: A multi-grained state space model with self-evolution regularization for offline rl. *Advances in neural information processing systems*, 37:22827–22849, 2024.

[46] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 2978–2988, 2019.

[47] Aviral Kumar, Aurick Zhou, George Tucker, and Sergey Levine. Conservative q-learning for offline reinforcement learning. *Advances in neural information processing systems*, 33: 1179–1191, 2020.

[48] Egor Cherepanov, Alexey Staroverov, Dmitry Yudin, Alexey K Kovalev, and Aleksandr I Panov. Recurrent action transformer with memory. *arXiv preprint arXiv:2306.09459*, 2023.

[49] Marcel Torne, Andy Tang, Yuejiang Liu, and Chelsea Finn. Learning long-context diffusion policies via past-token prediction. *arXiv preprint arXiv:2505.09561*, 2025.

[50] Longxin Kou, Fei Ni, Yan Zheng, Jinyi Liu, Yifu Yuan, Zibin Dong, and Jianye Hao. Kisa: A unified keyframe identifier and skill annotator for long-horizon robotics demonstrations. In *Forty-first International Conference on Machine Learning*, 2024.

[51] BinXu Wu, TengFei Zhang, Chen Yang, JiaHao Wen, HaoCheng Li, JingTian Ma, Zhen Chen, and JingYuan Wang. Sage: State-aware guided end-to-end policy for multi-stage sequential tasks via hidden markov decision process. *arXiv preprint arXiv:2509.19853*, 2025.

[52] Yipeng Chen, Wentao Tan, Lei Zhu, Fengling Li, Jingjing Li, Guoli Yang, and Heng Tao Shen. Non-markovian long-horizon robot manipulation via keyframe chaining. *arXiv preprint arXiv:2603.01465*, 2026.

[53] Max Sobol Mark, Jacky Liang, Maria Attarian, Chuyuan Fu, Debidatta Dwibedi, Dhruv Shah, and Aviral Kumar. Bpp: Long-context robot imitation learning by focusing on key history frames. *arXiv preprint arXiv:2602.15010*, 2026.

[54] Burak Han Demirbilek, Alper Demir, and Faruk Polat. External visual memory with autoencoder-based intrinsic motivation for reinforcement learning under partial observability. In *2025 IEEE 37th International Conference on Tools with Artificial Intelligence (ICTAI)*, pages 796–800. IEEE, 2025.

[55] Geraud Nangue Tasse, Matthew Riemer, Benjamin Rosman, and Tim Klinger. Beyond sliding windows: Learning to manage memory in non-markovian environments. *arXiv preprint arXiv:2512.19154*, 2025.

[56] Preetum Nakkiran, Gal Kaplun, Dimitris Kalimeris, Tristan Yang, Benjamin L Edelman, Fred Zhang, and Boaz Barak. Sgd on neural networks learns functions of increasing complexity. *arXiv preprint arXiv:1905.11604*, 2019.

[57] Harshay Shah, Kaustav Tamuly, Aditi Raghunathan, Prateek Jain, and Praneeth Netrapalli. The pitfalls of simplicity bias in neural networks. *Advances in Neural Information Processing Systems*, 33:9573–9585, 2020.

[58] Hojoon Lee, Dongyoon Hwang, Donghu Kim, Hyunseung Kim, Jun Jet Tai, Kaushik Subramanian, Peter R Wurman, Jaegul Choo, Peter Stone, and Takuma Seno. Simba: Simplicity bias for scaling up parameters in deep reinforcement learning. *arXiv preprint arXiv:2410.09754*, 2024.

[59] Vladimir Vapnik. *The nature of statistical learning theory*. Springer science & business media, 2013.

[60] Thomas M Cover. *Elements of information theory*. John Wiley & Sons, 1999.

[61] Tilmann Gneiting and Adrian E Raftery. Strictly proper scoring rules, prediction, and estimation. *Journal of the American statistical Association*, 102(477):359–378, 2007.

[62] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.

[63] Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT press Cambridge, 2016.

[64] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL `http://jmlr.org/papers/v22/20-1364.html`.

[65] Giacomo De Palma, Bobak Kiani, and Seth Lloyd. Random deep neural networks are biased towards simple functions. *Advances in Neural Information Processing Systems*, 32, 2019.

[66] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[67] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[68] Tianwei Ni, Michel Ma, Benjamin Eysenbach, and Pierre-Luc Bacon. When do transformers shine in rl? decoupling memory from credit assignment. *Advances in Neural Information Processing Systems*, 36:50429–50452, 2023.

[69] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: visual explanations from deep networks via gradient-based localization. *International journal of computer vision*, 128(2):336–359, 2020.

[70] Aditya Chattopadhay, Anirban Sarkar, Prantik Howlader, and Vineeth N Balasubramanian. Grad-cam++: Generalized gradient-based visual explanations for deep convolutional networks. In *2018 IEEE winter conference on applications of computer vision (WACV)*, pages 839–847. IEEE, 2018.

[71] Ajay Sridhar, Jennifer Pan, Satvik Sharma, and Chelsea Finn. Memer: Scaling up memory for robot control via experience retrieval. *arXiv preprint arXiv:2510.20328*, 2025.

[72] Marcel Torne, Karl Pertsch, Homer Walke, Kyle Vedder, Suraj Nair, Brian Ichter, Allen Z Ren, Haohuan Wang, Jiaming Tang, Kyle Stachowicz, et al. Mem: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.

[73] Jianbo Chen, Le Song, Martin Wainwright, and Michael Jordan. Learning to explain: An information-theoretic perspective on model interpretation. In *International conference on machine learning*, pages 883–892. PMLR, 2018.

[74] Jinsung Yoon, James Jordon, and Mihaela Van der Schaar. Invase: Instance-wise variable selection using neural networks. In *International conference on learning representations*, 2018.

[75] Yunchu Zhang, Shubham Mittal, Zhengyu Zhang, Liyiming Ke, Siddhartha Srinivasa, and Abhishek Gupta. Atk: Automatic task-driven keypoint selection for robust policy learning. *arXiv preprint arXiv:2506.13867*, 2025.

Appendix

A Proofs **17**

B Algorithms **22**

B.1 Buffer priority queue 23

B.2 Proxy policy training 23

B.3 Selector RL environment 24

B.4 End-to-end training and inference 25

C Synthetic and Grid Domain Details **26**

C.1 Synthetic environments 26

C.2 Grid environments 27

C.3 Training dataset 28

C.4 Training Keyframe Mnemonics policies 28

C.5 Baselines and benchmarking details 29

C.6 Parameter-budget comparison 30

C.7 Benchmark result analysis 30

C.8 Hardware and wall-clock 31

D Robot Manipulation Domain Details **31**

D.1 `MIKASA-Robo` environments 31

D.2 Pipeline overview 31

D.3 Architectures and parameter counts 31

D.4 Optimizer and schedule 32

D.5 Stage-1: Proxy policy f_θ 32

D.6 Stage-2: Selector policy π_{s} 33

D.7 Stage-3: BC policy π_{BC} 33

D.8 Camera-image data augmentation 33

D.9 `MIKASA-Robo` all results 33

D.10 Hardware and wall-clock 33

E Experimental Details **34**

E.1 Horizon-invariance experiments on synthetic and grid domains 34

E.2 Horizon-invariance experiment on `RememberColor3-`ΔT 34

E.3 Reproducibility 36

F Ablation Study **36**

F.1 Learned selector against a random selector 36

F.2 Selector discount factor 36

F.3 Surprise coefficient 37

F.4 Current observation in proxy training . . . 37
F.5 Queue tie-break rule . . . 37
F.6 Memory buffer size . . . 38

G Additional Analysis . . . 38
G.1 Interpretability of discovered keyframes . . . 38
G.2 Selector priority behavior . . . 38
G.3 BC Policy activation map . . . 39
G.4 Sensitivity to memory budget k . . . 40
G.5 PPO reward plots . . . 41
G.6 Limitation discussion . . . 41

H Extended Related Works . . . 42

A Proofs

Lemma 1 (Combinatorial cardinality of the sample space). *The total cardinality M of the space of distinct timestep-indexed training pairs $(\mathcal{B}_t, a_t)$ is given by:*

$$M = \sum_{i=1}^{N} \left(T_i + \sum_{j=1}^{k} \binom{T_i + 1}{j + 1} \right).$$

Proof. Assume a stochastic training procedure that samples a trajectory τ_i, a timestep $t \in [1, T_i]$, and a mask Ψ_k selecting a subset of $j \leq k$ observations from the history h_t without replacement. The masked-sampling process generates an observation subset representing the buffer $\mathcal{B}$ by selecting j observation frames, where $j \sim \min(\text{Binomial}(t, 0.5), k)$. Because this spans $j \in \{0, \ldots, \min(t, k)\}$, the sample space includes all subsets of size up to k, including the empty set for $j = 0$.

For any timestep t, the number of unique combinations of size $j \geq 1$ is $\binom{t}{j}$ evaluating to 0 if $t < j$. Summing these over all timesteps t yields the total combinations for size j:

$$\sum_{t=1}^{T_i} \binom{t}{j} = \binom{T_i + 1}{j + 1},$$

which simplifies via the combinatorial Hockey-stick identity [38].

For the case where $j = 0$, there is exactly $\binom{t}{0} = 1$ unique combination at each timestep. Summing this across all T_i timesteps adds exactly T_i combinations:

$$\sum_{t=1}^{T_i} \binom{t}{0} = T_i.$$

Summing the combinations for all possible mask sizes $j \in \{0, \ldots, k\}$ gives the sample space cardinality for a single trajectory:

$$M_i = T_i + \sum_{j=1}^{k} \binom{T_i + 1}{j + 1}.$$

Finally, summing across all N independent trajectories in $\mathcal{D}$ yields the total cardinality M of the training sample space:

$$M = \sum_{i=1}^{N} \left(T_i + \sum_{j=1}^{k} \binom{T_i + 1}{j + 1} \right).$$

□

Assumption 3 (Realizability of conditional programs by interpolating hypotheses). For $Z \in \{\text{key}, \text{bg}\}$, let $f_Z^* = \arg\min_{f \in \mathcal{H}_Z : f(\mathcal{X}_{Z,\ell}) = A_\ell \ \forall \ell} K(f)$ denote the minimum-complexity interpolating hypothesis using input source Z. We assume that the shortest conditional program mapping the training input sequence $\mathcal{X}_{Z,1:M}$ to the action sequence $A_{1:M}$ is realizable by the hypothesis class $\mathcal{H}$ up to an additive constant: $|K(f_Z^*) - K(A_{1:M} \mid \mathcal{X}_{Z,1:M})| \leq c_Z$, where c_Z is an overhead constant independent of the sample size M. This assumption combines the standard interpretation of conditional Kolmogorov complexity as the shortest program mapping inputs to outputs [32, 33] with the finite-sample expressivity of overparameterized neural networks, which can interpolate or memorize arbitrary training labels [34, 27, 35].

Theorem 1 (Expected gap in hypothesis complexity). *Under the assumption that training pairs are sampled i.i.d. from a sufficiently large support of size M (Lemma 1), the expected Kolmogorov complexity of the minimum-complexity background-based hypothesis f_{bg}^* is strictly greater than that of the keyframe-based hypothesis f_{key}^*.*

$$\mathbb{E}[K(f_{\text{bg}}^*)] > \mathbb{E}[K(f_{\text{key}}^*)].$$

Proof. We first analyze complexity using established equivalence results from algorithmic information theory [32]. Let $P(X, Y)$ denote a computable joint probability distribution generating data pairs (X, Y) with marginal entropy $H(Y)$.

According to Theorem 10 in Grünwald et al. [32], the expected Kolmogorov complexity $K(\cdot)$ of a random variable equals its Shannon entropy up to an additive term bounded by the algorithmic complexity of the data-generating distribution itself:

$$\mathbb{E}[K(Y)] = H(Y) + O(K(P)). \tag{6}$$

Similarly, Theorem 11 in Grünwald et al. [32] establishes that expected algorithmic mutual information $I(Y : X)$ equals the probabilistic Shannon mutual information $I(Y; X)$ up to an additive term bounded by the same algorithmic complexity:

$$\mathbb{E}[I(Y : X)] = I(Y; X) + O(K(P)). \tag{7}$$

The algorithmic conditional complexity is defined via algorithmic mutual information (Equation 20 in Grünwald et al. [32]) as $K(y \mid x^*) = K(y) - I(y : x)$. Taking the expectation over the data distribution and substituting Equations 6 and 7, we derive expected conditional Kolmogorov complexity in terms of conditional Shannon entropy [2]:

$$\begin{aligned} \mathbb{E}[K(Y)] - \mathbb{E}[I(Y : X)] &= H(Y) - I(Y; X) + O(K(P)), \\ \mathbb{E}[K(Y \mid X)] &= H(Y \mid X) + O(K(P)). \end{aligned} \tag{8}$$

We now apply this to our setting, where $P = P(\mathcal{D})$ denotes the distribution induced by the POMDP and expert policy that generate dataset $\mathcal{D}$, and we take $K(\mathcal{D}) = K(P(\mathcal{D}))$ to represent the constant algorithmic overhead of this data-generating process. During training, $(\mathcal{X}, A)$ pairs are sampled from a finite support of cardinality M induced by the masked-sampling procedure (Lemma 1) [3]. Extending Equation 8 to the entire sequence of M samples, the expected conditional Kolmogorov complexity of the sequence of actions given the sequence of sampled frames is bounded by the joint Shannon entropy plus the one-time algorithmic overhead of the data-generating distribution:

$$\mathbb{E}[K(A_{1:M} \mid \mathcal{X}_{1:M})] = H(A_{1:M} \mid \mathcal{X}_{1:M}) + O(K(\mathcal{D})). \tag{9}$$

Because the M sampled training pairs are treated as i.i.d., the joint conditional Shannon entropy scales linearly, $H(A_{1:M} \mid \mathcal{X}_{1:M}) = M \cdot H(A \mid \mathcal{X})$. Substituting this yields:

$$\mathbb{E}[K(A_{1:M} \mid \mathcal{X}_{1:M})] = M \cdot H(A \mid \mathcal{X}) + O(K(\mathcal{D})). \tag{10}$$

[2] For notational simplicity, we suppress the distinction between conditioning on X and on its shortest program X^*, absorbing the resulting additive terms into the $O(\cdot)$ overhead.

[3] Note that we use $\mathcal{X}$ to denote the input domain of interpolating hypotheses $f \in \mathcal{H}$, while $\mathcal{B}$ denotes a buffer subset used as input to the neural network f_θ; in both cases, the input size is bounded by k.

The conditional Kolmogorov complexity $K(y \mid x)$ is strictly defined as the length of the shortest program on a universal conditional prefix machine that outputs y when x is provided as input [32]. Therefore, for any interpolating hypothesis f_Z satisfying $f_Z(\mathcal{X}_{Z,\ell}) = A_\ell$ for all ℓ, where $Z \in \{\mathrm{key}, \mathrm{bg}\}$, there exists a fixed evaluator program which, given a description of f_Z and the input sequence $\mathcal{X}_{Z,1:M}$, perfectly reconstructs the action sequence $A_{1:M}$. Hence, the conditional sequence complexity is upper-bounded by the hypothesis complexity:

$$K(A_{1:M} \mid \mathcal{X}_{Z,1:M}) \leq K(f_Z^*) + O(1).$$

Assumption 3 states that, for each input source $Z \in \{\mathrm{key}, \mathrm{bg}\}$, the minimum-complexity interpolating hypothesis f_Z^* realizes the corresponding shortest conditional program up to an additive constant:

$$|K(f_Z^*) - K(A_{1:M} \mid \mathcal{X}_{Z,1:M})| \leq c_Z.$$

Equivalently, unpacking the absolute value gives the two-sided bound

$$K(A_{1:M} \mid \mathcal{X}_{Z,1:M}) - c_Z \leq K(f_Z^*) \leq K(A_{1:M} \mid \mathcal{X}_{Z,1:M}) + c_Z.$$

To analyze the complexity gap $K(f_{\mathrm{bg}}^*) - K(f_{\mathrm{key}}^*)$, we derive a conservative lower bound by using a lower bound for the background hypothesis and an upper bound for the keyframe hypothesis. This yields

$$K(f_{\mathrm{bg}}^*) \geq K(A_{1:M} \mid \mathcal{X}_{\mathrm{bg},1:M}) - c_{\mathrm{bg}},$$
$$K(f_{\mathrm{key}}^*) \leq K(A_{1:M} \mid \mathcal{X}_{\mathrm{key},1:M}) + c_{\mathrm{key}}.$$

Subtracting the keyframe upper bound from the background lower bound, and taking the expectation over the data-generating distribution, yields the expected hypothesis complexity gap:

$$\mathbb{E}[K(f_{\mathrm{bg}}^*)] - \mathbb{E}[K(f_{\mathrm{key}}^*)] \geq \mathbb{E}[K(A_{1:M} \mid \mathcal{X}_{\mathrm{bg},1:M})] - \mathbb{E}[K(A_{1:M} \mid \mathcal{X}_{\mathrm{key},1:M})] - (c_{\mathrm{key}} + c_{\mathrm{bg}}).$$

Substituting the sequence complexities from Equation 8 and 10, we obtain:

$$\mathbb{E}[K(f_{\mathrm{bg}}^*)] - \mathbb{E}[K(f_{\mathrm{key}}^*)] \geq M\big(I(A; \mathcal{X}_{\mathrm{key}}) - I(A; \mathcal{X}_{\mathrm{bg}})\big) - O(K(\mathcal{D})) - (c_{\mathrm{key}} + c_{\mathrm{bg}}).$$

Under Assumption 2, the mutual information gap $I(A; \mathcal{X}_{\mathrm{key}}) - I(A; \mathcal{X}_{\mathrm{bg}})$ is strictly positive. Because the sample support M is combinatorially massive (Lemma 1), this linear scaling term strictly dominates the combined constant overheads $O(K(\mathcal{D})) + (c_{\mathrm{key}} + c_{\mathrm{bg}})$. Thus, for sufficiently large M, we obtain:

$$\mathbb{E}[K(f_{\mathrm{bg}}^*)] > \mathbb{E}[K(f_{\mathrm{key}}^*)].$$

□

Assumption 4 (Simplicity-biased SGD approximation). Conditioned on reaching an interpolating solution in $\mathcal{H}^*$, SGD is approximated as selecting hypotheses with probability biased toward lower Kolmogorov complexity:

$$P_{\mathrm{SGD}}(f \mid \mathcal{D}) \propto 2^{-K(f)}, \qquad f \in \mathcal{H}^*.$$

This approximation follows the simplicity-bias view of neural networks, where low-complexity functions receive larger prior mass [1, 2, 36].

Proposition 1 (Conditional simplicity-bias preference). *Under the simplicity-bias assumption and the complexity gap established in Theorem 1, Stochastic Gradient Descent (SGD) is biased toward the invariant hypothesis class $\mathcal{H}_{\mathrm{in}}$. Given a masking-induced sample distribution with support size M (Lemma 1), the probability of selecting an invariant hypothesis is approximated by*

$$P_{\mathrm{SGD}}(\mathcal{H}_{\mathrm{in}} \mid \mathcal{D}) \approx \frac{1}{1 + 2^{-\left[M\left(I(A;\mathcal{X}_{\mathrm{key}}) - I(A;\mathcal{X}_{\mathrm{bg}})\right) - \mathcal{O}(1)\right]}}.$$

Proof. Under the Empirical Risk Minimization (ERM) framework [59], an overparameterized neural network can realize hypotheses in the interpolating set $\mathcal{H}^*$ that achieve zero empirical risk on $\mathcal{D}$. To analyze which interpolating hypothesis is selected by SGD, we use the simplicity-bias approximation in Assumption 4, motivated by evidence that neural network function priors and SGD solutions are biased toward low-complexity functions [1, 2, 36]. The probability of selecting an invariant hypothesis is obtained by marginalizing over the invariant class:

$$P_{\mathrm{SGD}}(\mathcal{H}_{\mathrm{in}} \mid \mathcal{D}) \approx \frac{\sum_{f \in \mathcal{H}_{\mathrm{in}}} 2^{-K(f)}}{\sum_{f \in \mathcal{H}^*} 2^{-K(f)}}.$$

Since $\mathcal{H}^*$ is partitioned into the invariant class and the non-invariant class,

$$\mathcal{H}^* = \mathcal{H}_{\text{in}} \cup \mathcal{H}_{\neg\text{in}},$$

we can rewrite this as

$$P_{\text{SGD}}(\mathcal{H}_{\text{in}} \mid \mathcal{D}) \approx \frac{\sum_{f \in \mathcal{H}_{\text{in}}} 2^{-K(f)}}{\sum_{f \in \mathcal{H}_{\text{in}}} 2^{-K(f)} + \sum_{f \in \mathcal{H}_{\neg\text{in}}} 2^{-K(f)}}.$$

Because the probability mass decays exponentially with complexity, we use the standard minimum-description approximation that each class-level mass is dominated, up to constant factors independent of M, by its minimum-complexity interpolating hypothesis. Let

$$f^*_{\text{key}} = \arg\min_{f \in \mathcal{H}_{\text{in}}} K(f), \qquad f^*_{\text{bg}} = \arg\min_{f \in \mathcal{H}_{\neg\text{in}}} K(f).$$

Then

$$\sum_{f \in \mathcal{H}_{\text{in}}} 2^{-K(f)} \approx 2^{-K(f^*_{\text{key}})}, \qquad \sum_{f \in \mathcal{H}_{\neg\text{in}}} 2^{-K(f)} \approx 2^{-K(f^*_{\text{bg}})}.$$

Substituting this approximation gives

$$P_{\text{SGD}}(\mathcal{H}_{\text{in}} \mid \mathcal{D}) \approx \frac{2^{-K(f^*_{\text{key}})}}{2^{-K(f^*_{\text{key}})} + 2^{-K(f^*_{\text{bg}})}}.$$

Dividing numerator and denominator by $2^{-K(f^*_{\text{key}})}$ gives

$$P_{\text{SGD}}(\mathcal{H}_{\text{in}} \mid \mathcal{D}) \approx \frac{1}{1 + 2^{-\left(K(f^*_{\text{bg}}) - K(f^*_{\text{key}})\right)}}.$$

By Theorem 1, the expected complexity gap between the non-invariant/background-based hypothesis and the invariant/keyframe-based hypothesis satisfies

$$\mathbb{E}[K(f^*_{\text{bg}})] - \mathbb{E}[K(f^*_{\text{key}})] \geq M\big(I(A; \mathcal{X}_{\text{key}}) - I(A; \mathcal{X}_{\text{bg}})\big) - \mathcal{O}(1).$$

Here, the constant overhead $O(K(\mathcal{D})) + c_{\text{key}} + c_{\text{bg}}$ is absorbed into the $\mathcal{O}(1)$ term. Therefore, for a typical dataset sampled from the masking-induced distribution, we use the corresponding expected gap approximation

$$K(f^*_{\text{bg}}) - K(f^*_{\text{key}}) \approx M\big(I(A; \mathcal{X}_{\text{key}}) - I(A; \mathcal{X}_{\text{bg}})\big) - \mathcal{O}(1).$$

Substituting this into the class-probability expression yields

$$P_{\text{SGD}}(\mathcal{H}_{\text{in}} \mid \mathcal{D}) \approx \frac{1}{1 + 2^{-\left[M\left(I(A;\mathcal{X}_{\text{key}}) - I(A;\mathcal{X}_{\text{bg}})\right) - \mathcal{O}(1)\right]}}.$$

Thus, when the keyframe information advantage is positive and the assumptions hold, the simplicity-bias approximation predicts a rapid increase in the probability of selecting an invariant hypothesis. This follows because the exponent scales linearly with the combinatorially large support size M. [4] □

Corollary 1 (Keyframe recovery from the frozen loss objective). *Let us define a subset-selection objective function*

$$J_{\theta^*}(\mathcal{B}) = \mathbb{E}[\mathcal{L}(f_{\theta^*}(\mathcal{B}), A)]$$

over a frozen predictor f_{θ^*} obtained under Proposition 1 after training with negative log-likelihood loss $\mathcal{L}$, where f_{θ^*} represents a near-optimal invariant hypothesis in $\mathcal{H}_{\text{in}}$. Consider candidate buffer subsets $\mathcal{C}_k = \{\mathcal{B}_t \subseteq h_t : |\mathcal{B}_t| \leq k\}$, and let $\mathcal{B}_{\text{opt}} \in \arg\min_{\mathcal{B} \in \mathcal{C}_k} J_{\theta^*}(\mathcal{B})$. If keyframe subsets $\mathcal{B} \subset \mathcal{X}_{\text{key}}$ have a positive action-information margin over all non-keyframe candidate subsets, and this margin dominates the approximation error of the frozen predictor f_{θ^*}, then the loss objective $J_{\theta^*}(\mathcal{B})$ recovers the keyframes up to information-equivalent subsets. In particular, if $\mathcal{X}_{\text{key}}$ is unique under the memory budget k, then $\mathcal{B}_{\text{opt}} = \mathcal{X}_{\text{key}}$.

Proof. For negative log-likelihood loss, the Bayes-optimal expected loss conditioned on an input buffer subset $\mathcal{B}$ is the conditional entropy, since the logarithmic score is a strictly proper scoring rule

[4] If the assumptions fail, for example, when a lower-complexity non-invariant shortcut exists, this preference need not hold.

and the expected negative log-likelihood decomposes into conditional entropy plus a nonnegative KL divergence [60, 61]:

$$\inf_f \mathbb{E}[\mathcal{L}(f_{\theta^*}(\mathcal{B}), A)] = H(A \mid \mathcal{B}).$$

Using the Shannon identity

$$H(A \mid \mathcal{B}) = H(A) - I(A; \mathcal{B})$$

minimizing the Bayes-optimal negative log-likelihood is equivalent to maximizing the mutual information between the selected subset and the action [60]. Define the action-information margin of the keyframe set over competing candidate subsets as

$$\mu_k = \inf_{\mathcal{B} \in \mathcal{C}_k : \mathcal{B} \not\sim \mathcal{X}_{\text{key}}} \left[I(A; \mathcal{X}_{\text{key}}) - I(A; \mathcal{B}) \right],$$

where $\mathcal{B} \sim \mathcal{X}_{\text{key}}$ denotes an information-equivalent subset. The condition $\mu_k > 0$ extends Assumption 2 from the background subset to the full candidate search space $\mathcal{C}_k$, requiring that keyframes have strictly larger action information than any non-equivalent competing subset.

Since Proposition 1 biases training toward the invariant class, suppose the frozen model f_{θ^*} is a near-optimal keyframe-conditioned predictor. Define its approximation error on the keyframe input as

$$\epsilon_{\text{app}} = J_{\theta^*}(\mathcal{X}_{\text{key}}) - H(A \mid \mathcal{X}_{\text{key}}).$$

Since negative log-likelihood risk is lower-bounded by conditional entropy, $\epsilon_{\text{app}} \geq 0$. Thus,

$$J_{\theta^*}(\mathcal{X}_{\text{key}}) = H(A \mid \mathcal{X}_{\text{key}}) + \epsilon_{\text{app}}.$$

For any competing subset $\mathcal{B} \in \mathcal{C}_k$ with $\mathcal{B} \not\sim \mathcal{X}_{\text{key}}$, every predictor using only $\mathcal{B}$ has negative log-likelihood risk at least its Bayes risk by nonnegativity of the KL divergence [60, 61]:

$$J_{\theta^*}(\mathcal{B}) \geq H(A \mid \mathcal{B}).$$

By the definition of μ_k,

$$I(A; \mathcal{X}_{\text{key}}) - I(A; \mathcal{B}) \geq \mu_k,$$

which is equivalent to

$$H(A \mid \mathcal{B}) \geq H(A \mid \mathcal{X}_{\text{key}}) + \mu_k.$$

Therefore,

$$J_{\theta^*}(\mathcal{B}) \geq H(A \mid \mathcal{X}_{\text{key}}) + \mu_k.$$

If the information margin dominates the approximation error,

$$\mu_k > \epsilon_{\text{app}},$$

then

$$J_{\theta^*}(\mathcal{X}_{\text{key}}) = H(A \mid \mathcal{X}_{\text{key}}) + \epsilon_{\text{app}} < H(A \mid \mathcal{X}_{\text{key}}) + \mu_k \leq J_{\theta^*}(\mathcal{B})$$

for every non-equivalent competing subset $\mathcal{B}$. Hence, the minimizer of the frozen loss objective belongs to the keyframe equivalence class. If $\mathcal{X}_{\text{key}}$ is unique under the memory budget k, then

$$\mathcal{B}_{\text{opt}} = \mathcal{X}_{\text{key}}.$$

This keyframe recovery guarantee naturally extends to continuous action spaces optimized via Mean Squared Error (MSE). Under the assumption that the conditional distribution of the actions is Gaussian with a fixed variance, minimizing the MSE is equivalent to minimizing the negative log-likelihood up to affine constants [62, 63]. Consequently, the same argument applies with conditional differential entropy and mutual information replacing their discrete counterparts, so the frozen MSE objective recovers $\mathcal{X}_{\text{key}}$ under the same positive-margin and approximation-error conditions. □

Theorem 2 (Infinite-horizon keyframe retention). *Assume a POMDP in which the expert behavior strictly satisfies Assumptions 1 and 2. Let π_{s}^* be an optimal selector minimizing the frozen objective $J_{\theta^*}(\mathcal{B}_t)$ (Corollary 1). Suppose a unique keyframe set $\mathcal{X}_{\text{key}}$ exists that has the highest mutual information with the action and fits within the memory budget k. Then, as $T \to \infty$, π_{s}^* assigns higher priority to keyframes than to background frames, ensuring that the priority queue retains the keyframe set for the entire horizon T without eviction, i.e., $\mathcal{B}_T = \mathcal{X}_{\text{key}}$.*

Proof. We analyze the behavior of an optimal selector policy π_{s}^* over the temporal horizon T. The reward function $R(\mathcal{B}_t)$ is designed to increase as the frozen loss objective $J_{\theta^*}(\mathcal{B}_t)$ decreases. Thus,

maximizing expected return requires π_{s}^* to assign priority scores that configure the buffer $\mathcal{B}_t$ into the subset minimizing $J_{\theta^*}(\mathcal{B})$.

By Assumption 2 and Corollary 1, the keyframe set $\mathcal{X}_{\mathrm{key}}$ has a strictly positive action-information margin $\mu_k > 0$ over all non-keyframe subsets. Consider a timestep t at which the keyframe set has already been inserted into the buffer. Let a new background observation $o_t \in \mathcal{X}_{\mathrm{bg}}$ be evaluated. If π_{s}^* assigns a priority p_t high enough to evict a keyframe, then the resulting buffer $\mathcal{B}_t$ has strictly lower mutual information with the action, decreasing by at least μ_k.

By Corollary 1, this induces a strictly higher expected loss:

$$J_{\theta^*}(\mathcal{B}_t) \geq H(A \mid \mathcal{X}_{\mathrm{key}}) + \mu_k > J_{\theta^*}(\mathcal{X}_{\mathrm{key}}),$$

which penalizes the reward. Therefore, any such eviction is suboptimal, and π_{s}^* must assign lower priority to background observations than to keyframes. Once the keyframe set $\mathcal{X}_{\mathrm{key}}$ is fully inserted into the buffer, this argument applies recursively. Hence, under the optimal selector policy, background observations never receive sufficient priority to evict keyframes. Thus, the priority queue preserves $\mathcal{X}_{\mathrm{key}}$ indefinitely, yielding $\mathcal{B}_T = \mathcal{X}_{\mathrm{key}}$. □

B Algorithms

Algorithm 2 FIFO priority queue — training-time eviction rule.

Parameters: training memory budget k, rejection threshold η_{r}
State: buffer $\mathcal{B}_t \leftarrow \emptyset$, insertion counter $c \leftarrow 0$
procedure PUSH(observation o, priority p)
    if $p < \eta_{\mathrm{r}}$ **then**
        return ▷ reject priority below threshold
    end if
    $c \leftarrow c + 1$ ▷ monotone insertion timestamp
    if $|\mathcal{B}_t| < k$ **then**
        $\mathcal{B}_{t+1} \leftarrow \mathcal{B}_t \cup \{(o, p, c)\}$ ▷ free slot, insert directly
    else
        $(o^\star, p^\star, c^\star) \leftarrow \arg\min_{(o', p', c') \in \mathcal{B}_t} (p',\ c')$ ▷ lowest priority; *oldest* on a tie
        if $p \geq p^\star$ **then**
            $\mathcal{B}_{t+1} \leftarrow \big(\mathcal{B}_t \setminus \{(o^\star, p^\star, c^\star)\}\big) \cup \{(o, p, c)\}$
        end if
    end if
end procedure

Algorithm 3 LIFO priority queue — inference-time eviction rule.

Parameters: inference memory budget k, rejection threshold η_{r}, no-repeat threshold η_{n}
State: buffer $\mathcal{B}_t \leftarrow \emptyset$, insertion counter $c \leftarrow 0$, last priority $p_{\mathrm{last}} \leftarrow 0$
procedure PUSH(observation o, priority p)
    if $p < \eta_{\mathrm{r}}$ **or** $|p - p_{\mathrm{last}}| < \eta_{\mathrm{n}}$ **then**
        return ▷ reject low or quasi-repeated priority
    end if
    $c \leftarrow c + 1$
    if $|\mathcal{B}_t| < k$ **then**
        $\mathcal{B}_{t+1} \leftarrow \mathcal{B}_t \cup \{(o, p, c)\}$
    else
        $(o^\star, p^\star, c^\star) \leftarrow \arg\min_{(o', p', c') \in \mathcal{B}_t} (p',\ -c')$ ▷ lowest priority; *newest* on a tie
        if $p \geq p^\star$ **then**
            $\mathcal{B}_{t+1} \leftarrow \big(\mathcal{B}_t \setminus \{(o^\star, p^\star, c^\star)\}\big) \cup \{(o, p, c)\}$
        end if
    end if
    $p_{\mathrm{last}} \leftarrow p$ ▷ update even if no eviction occurred
end procedure

B.1 Buffer priority queue

The memory buffer $\mathcal{B}_t$ introduced in Section 2.2 is realised by a bounded priority queue of capacity k. The buffer design supports two key behaviors: (i) *overflow forgetting*, where consistently assigning high priorities causes queue overflow and removes keyframes from the buffer, and (ii) *background rejection*, where assigning priorities below a rejection threshold actively ignores background observations. Both training and inference queue use the same insertion semantics - the selector emits a priority $p \in [0, 1]$ per observation, and the queue accepts or rejects based on p; the two phases differ *only* in how ties on the lowest priority are broken at eviction time. Both preserve the temporal ordering of frames in the buffer. Algorithms 2 and 3 state the two rules. We use $\eta_{\mathrm{r}} = 0.5$ as the rejection threshold throughout, and $\eta_{\mathrm{n}} = 0.05$ as the no-repeat threshold for the inference queue.

Why FIFO at training time. The training rule (Algorithm 2) breaks ties in favour of evicting the *oldest* entry. Under this rule, a frame that has been assigned a uniformly high priority will still age out as new high-priority frames arrive, so the selector cannot rely on saturating priorities to retain information. This sharpens the gradient signal driving π_{s} towards genuinely discriminative scores rather than constant-1 outputs.

Why LIFO at inference time. The inference rule (Algorithm 3) breaks ties by evicting the *newest* entry. Thus, initial frames assigned high priority by the selector are preserved, enabling robust keyframe retention and maintaining the keyframe set over arbitrarily long horizons. The no-repeat threshold η_{n} further prevents near-tied priority observations from entering the queue, ensuring that it retains diversity. This especially helps with retaining keyframes when there exists multiple information-equivalent keyframe subsets in the demonstration dataset.

Note that under a near-optimal selector and a sufficient memory budget k (Theorem 2), such that the buffer can store all true keyframes, both queues exhibit identical behavior. We observed this empirically in synthetic domains, where the discovered mnemonics remained consistent regardless of the queue strategy used during inference.

B.2 Proxy policy training

Section 2.1 introduces the proxy policy $f_\theta : \mathcal{B} \to \mathcal{A}$ trained by masked-sampling of past observations (Algorithm 1). In practice, we implement two equivalent realizations of this training procedure, selected based on the scale of the masked-sampling support M (Lemma 1).

Online masked-sampling (robot manipulation). For `MIKASA-Robo` tasks we draw a fresh mask Ψ_k at every gradient step, as stated in Algorithm 1: a trajectory $\tau \sim \mathcal{D}$ and timestep $t \in \{1, \ldots, T_\tau\}$ are sampled, the masked-sampling distribution $n \sim \min(\mathrm{Binomial}(t, 0.5), k)$ selects a memory buffer $\mathcal{B}_t = \{o_i \in h_t \mid \Psi_k(i) = 1\}$ of cardinality $|\mathcal{B}_t| \leq k$, and the proxy regresses the expert action a_t under negative log-likelihood loss $\mathcal{L}$.

Offline masked-sampling rollouts (synthetic and grid domains). For the synthetic and grid domains (Add, Scattered-Copy, T-Maze, MiniGrid-Memory, LTMB-Hallway), the support size M is small enough to precompute masked samples. We therefore run multiple offline passes over $\mathcal{D}$ and cache the random buffer-action pairs on disk, rather than resampling masks at every gradient step. In each pass, for each trajectory $\tau_i \in \mathcal{D}$, we iterate over $t = 1, \ldots, T_i$, and execute the masked-sampling procedure (Algorithm 1) for each timestep. The resulting pairs $(\mathcal{B}_t, a_t)$ are written to the cache, and the proxy f_θ is trained for multiple epochs on the cached pairs. The number of passes trades off cache size against Monte Carlo coverage of the masked-sampling support and is chosen empirically so that the total number of cached pairs exceeds the sufficient support size M required by Theorem 1. Although online masked-sampling provides the maximum possible cardinality support M, this offline counterpart helped us with more controlled experiments on M to empirically verify Theorem 1 as shown in Figure 2. See Appendix C.3 for more details.

Lemma 2 (Equivalence to uniform selector priority queue). *Let $\mathcal{P}_{\mathrm{mask}}$ denote the distribution over buffer states $\mathcal{B}$ induced by masked-sampling (Algorithm 1), and let $\mathcal{P}_{\mathrm{queue}}$ denote the distribution induced by using a random selector π_{rand} that assigns uniform priorities $p \in [0, 1]$ to observations retained in a priority queue. Both procedures induce the same distribution over buffer states.*

Proof. Under $\mathcal{P}_{\text{mask}}$, the buffer is drawn by picking a size $n \sim \min(\text{Binomial}(t, 0.5), k)$ uniformly at random and then selecting n indices uniformly without replacement from $\{1, \ldots, t\}$ (Lemma 1). Under $\mathcal{P}_{\text{queue}}$, each past observation $o_i \in h_t$ receives an i.i.d. priority $p_i \sim \text{Unif}(0, 1)$, and the queue retains the top-k priorities exceeding the rejection threshold η_r. Because the priorities are exchangeable and independent of i, every k-subset of h_t is selected with equal probability conditional on its cardinality, and the cardinality distribution is $\min(\text{Binomial}(t, \Pr[p > \eta_r]), k)$. With $\eta_r = 0.5$ we have $\Pr[p > \eta_r] = 0.5$, recovering the masked-sampling cardinality distribution of Algorithm 1. Hence $\mathcal{P}_{\text{queue}} = \mathcal{P}_{\text{mask}}$ at the level of marginal over $\mathcal{B}_t$. □

The above result establishes an equivalence between the masked-sampling procedure (Algorithm 1) and selector-driven buffer construction under a random selector policy. Intuitively, this shows that the selector optimization (Section 2.2) operates over the same combinatorial space of buffer states explored during masked-sampling training. Consequently, the learned objective J_{θ^*} remains aligned with the optimization problem solved by the selector policy. Because J_{θ^*} wraps a function that learns a keyframe-conditioned hypothesis favored by Theorem 1, optimizing the selector is naturally biased toward recovering keyframe subsets.

B.3 Selector RL environment

We provide the algorithmic specification of the deterministic MDP $\langle \mathcal{O}, \mathcal{P}, \mathcal{K}, R, \gamma \rangle$ described in Section 2.2, on which the selector policy π_{s} is trained with PPO. Algorithm 4 states the per-step transition: the policy observes $o_t \in \mathcal{O}$, emits a priority $p_t \in \mathcal{P} = [0, 1]$, and the environment updates the priority queue $\mathcal{B}_t$ via the transition rule $\mathcal{K}$ (Algorithm 2). The post-update buffer is then scored by the frozen proxy f_{θ^*} to compute the surprise-weighted log-reward $R(\mathcal{B}_t)$ from Equation 5. The episode terminates after T_τ steps and the next trajectory is loaded from $\mathcal{D}$.

Algorithm 4 Selector RL environment step

Require: dataset $\mathcal{D}$, frozen proxy f_{θ^*}, memory budget k, normalisation constant λ, surprise coefficients $\{\beta_{\tau,t}\}$, queue eviction rule PUSH (Algorithm 2)
State: current trajectory $\tau \in \mathcal{D}$, step counter t, buffer $\mathcal{B}_{t-1}$

procedure STEP(o_t, p_t) ▷ $p_t \sim \pi_{\text{s}}(\cdot \mid o_t)$
  $a_t \leftarrow \tau$.ACTION(t) ▷ expert action from $\mathcal{D}$
  $\mathcal{B}_t \leftarrow$ PUSH($\mathcal{B}_{t-1}, o_t, p_t$) ▷ priority-queue update
  $\ell_t \leftarrow \mathcal{L}\big(f_{\theta^*}(\mathcal{B}_t), a_t\big)$ ▷ frozen proxy objective $J_{\theta^*}(\mathcal{B}_t)$
  $r_t \leftarrow \beta_{\tau,t} \cdot \dfrac{\log(\ell_t + \lambda)}{\log(\lambda)}$ ▷ Equation 5
  $t \leftarrow t + 1$
  if $t = T_\tau$ **then** ▷ episode terminated
    $\tau \sim \mathcal{D}$ ▷ load next trajectory
    $\mathcal{B}_{t-1} \leftarrow \emptyset$, $t \leftarrow 1$
    done $\leftarrow$ TRUE
  else
    done $\leftarrow$ FALSE
  end if
  $o_{t+1} \leftarrow \tau$.OBS(t)
  return $(o_{t+1}, r_t, \text{done})$
end procedure

The surprise coefficient $\beta_{\tau,t}$ in Equation 5 is precomputed once before PPO training as $\beta_{\tau,t} = T_\tau \cdot \text{softmax}_t(J^{\tau,t}_{\theta^*}/\kappa)$, where $J^{\tau,t}_{\theta^*} = \mathbb{E}_{\Psi_k}[\mathcal{L}(f_{\theta^*}(\mathcal{B}_t), a_t)]$ is the expected masked-sampling loss evaluated at (τ, t). Surprise scaling is applied on a per-domain basis: we enable it for synthetic and grid domains, where the loss landscape over t is sharply peaked at decision-critical timesteps, and disable it ($\beta_{\tau,t} \equiv 1$) for robot manipulation domains, where the unscaled reward in Equation 5 empirically provides a more stable PPO training signal. At training time, the environment is vectorized across n_{envs} parallel trajectories drawn from $\mathcal{D}$, and the proxy is evaluated on batches of buffer states for scalable RL training.

B.4 End-to-end training and inference

Algorithm 5 describes the end-to-end training procedure introduced in Sections 2.1–2.3. The proxy f_θ is trained using the masked-sampling algorithm (Algorithm 1), then frozen and used to score the reward $R(\mathcal{B}_t)$ for selector RL (Algorithm 4). The trained selector is then used to produce an augmented dataset $\mathcal{D}^\dagger$ by rolling π_{s} on $\mathcal{D}$ with the LIFO eviction rule[5], which is then used to train the BC policy $\pi_{\mathrm{BC}}(a_t \mid o_t, \mathcal{B}_t)$. See Appendix C and D for more details.

Algorithm 5 Keyframe Mnemonics training

Require: dataset $\mathcal{D}$, training memory budget k, proxy f_θ, selector π_{s}, BC policy π_{BC}, training epochs $E_f, E_{\mathrm{s}}, E_{\mathrm{BC}}$
Returns: trained $(f_\theta, \pi_{\mathrm{s}}, \pi_{\mathrm{BC}})$

`// Stage 1 - proxy policy training (Section 2.1)`
for $e = 1, \ldots, E_f$ **do**
  update θ on a batch sampled by Algorithm 1 on $\mathcal{D}$
end for
freeze $\theta^* \leftarrow \theta$; define $J_{\theta^*}(\mathcal{B}) = \mathbb{E}[\mathcal{L}(f_{\theta^*}(\mathcal{B}), a)]$
`// Stage 2 - selector policy training (Section 2.2)`
precompute surprise coefficients $\{\beta_{\tau,t}\}$ from J_{θ^*}
for $e = 1, \ldots, E_{\mathrm{s}}$ **do**
  roll out the selector MDP (Algorithm 4) with FIFO eviction
  update π_{s} by PPO on the surprise-weighted reward $R(\mathcal{B}_t)$
end for
`// Stage 3 - BC policy training (Section 2.3)`
$\mathcal{D}^\dagger \leftarrow \emptyset$
for $\tau \in \mathcal{D}$ **do**
  roll π_{s} on τ with LIFO eviction (Algorithm 3)
  append $\{(\mathcal{B}_t, o_t, a_t)\}_{t=1}^{T_\tau}$ to $\mathcal{D}^\dagger$
end for
for $e = 1, \ldots, E_{\mathrm{BC}}$ **do**
  update π_{BC} on a batch from $\mathcal{D}^\dagger$ under the BC loss
end for
return $(f_{\theta^*}, \pi_{\mathrm{s}}, \pi_{\mathrm{BC}})$

Algorithm 6 Keyframe Mnemonics inference

Require: environment $\mathcal{E}$, trained selector π_{s}, trained BC policy π_{BC}, training memory budget k, max rollout steps $T_{\max}$, queue eviction rule PUSH (Algorithm 3)

$o_1 \leftarrow \mathcal{E}.\textsc{Reset}()$
$\mathcal{B}_0 \leftarrow \emptyset$
for $t = 1, \ldots, T_{\max}$ **do**
  $p_t \leftarrow \pi_{\mathrm{s}}(o_t)$ ▷ assign priority
  $\mathcal{B}_t \leftarrow \textsc{Push}(\mathcal{B}_{t-1}, o_t, p_t)$ ▷ LIFO buffer update
  $a_t \leftarrow \pi_{\mathrm{BC}}(\mathcal{B}_t, o_t)$
  $o_{t+1} \leftarrow \mathcal{E}.\textsc{Step}(a_t)$
end for

At inference time the trained $(\pi_{\mathrm{s}}, \pi_{\mathrm{BC}})$ pair is rolled out in the target environment as specified in Algorithm 6. At each step the selector emits a priority $p_t = \pi_{\mathrm{s}}(o_t)$ that updates the buffer $\mathcal{B}_t$ under the LIFO eviction rule of Algorithm 3 (matching the rule used to construct $\mathcal{D}^\dagger$), and the BC policy emits an action $a_t = \pi_{\mathrm{BC}}(\mathcal{B}_t, o_t)$ that is applied to the environment for at most $T_{\max}$ steps.

[5]For synthetic domains where Theorem 2 holds strongly, we empirically observe that either FIFO or LIFO eviction can be used at inference time without affecting the discovered mnemonics; the LIFO rule is preferred for grid and robot manipulation domains where the assumptions hold less tightly.

C Synthetic and Grid Domain Details

C.1 Synthetic environments

In this section we show expert trajectories for the three synthetic memory benchmarks. Cell colors group conceptually related observations: / / mark the *keyframes* that an oracle selector must retain, marks background/noise observations that an optimal selector should reject, and marks the decision-critical query timestep at which the policy emits its action.

Add [28]. A sequence of T noisy real-valued observations. Two positions t_1, t_2 such that $t_1 < t_2$ are marked with binary flags (the "first $(1,0)$" and "second $(0,1)$" summands); at the final timestep $t = T-1$ both flags are simultaneously set $(1,1)$, signalling the policy to emit the sum of the two marked values. The per-step observation is the triple $o_t = (v_t,\ \mathbf{1}[t = t_1],\ \mathbf{1}[t = t_2])$ with $v_t \sim \mathcal{U}[0,1]$, and the target action is the scalar $a_t = v_{t_1} + v_{t_2}$ at $t = T-1$ and $a_t = 0$ elsewhere. Example with $T = 6, t_1 = 1, t_2 = 3$:

t	0	1	2	3	4	5
o_t	$(0.42, 0, 0)$	$(0.83, 1, 0)$	$(0.17, 0, 0)$	$(0.65, 0, 1)$	$(0.91, 0, 0)$	$(0.0, 1, 1)$
a_t	0	0	0	0	0	1.48

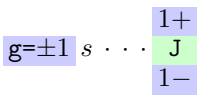
The three keyframes $\mathcal{X}_{\text{key}} = \{o_1, o_3, o_5\}$ comprise the two flag-marked summands (,) and the emit step ( ); the latter is itself a keyframe because the policy must use the emit token to know when to emit. The supervised target at the emit step is $a_5 = v_{t_1} + v_{t_2} = 0.83 + 0.65 = 1.48$; all other timesteps carry unmarked $(v_t, 0, 0)$ observations and are background frames. Evaluation is done on sequences with larger T obtained by increasing the number of unmarked $(v_t, 0, 0)$ background observations in the sequence.

Scattered-Copy [24]. A subsequence of n_{relevant} relevant tokens is scattered among n_{noise} noise tokens to form a source segment of length $L_{\text{src}} = n_{\text{relevant}} + n_{\text{noise}}$; the source is followed by a recall segment of n_{relevant} delimiter tokens `E` scattered among n_{blank} blank tokens □ (length $L_{\text{rec}} = n_{\text{relevant}} + n_{\text{blank}}$). At each delimiter the policy must emit the next relevant token in the order it originally appeared, and a no-op token `0` at every other position. The set of tokens for relevant and noise are disjoint and remains fixed for the task during both training and evaluation. Example with $n_{\text{relevant}} = 3, n_{\text{noise}} = 3, n_{\text{blank}} = 5$:

t	0	1	2	3	4	5	6	7	8	9	10	11	12	13
o_t	$\mathtt{N}_1$	A	$\mathtt{N}_2$	B	$\mathtt{N}_3$	C	□	□	E	□	E	□	□	E
a_t	0	0	0	0	0	0	0	0	A	0	B	0	0	C

The six keyframes $\mathcal{X}_{\text{key}} = \{o_1, o_3, o_5, o_8, o_{10}, o_{13}\}$ comprise the three relevant source tokens (/ /) and the three delimiter tokens ( ); the delimiters are themselves keyframes because the policy must use an `E` token to know it is at an emit position. The noise tokens $\mathtt{N}_i$ in the source and blank tokens in the recall are background frames. Evaluation is done by increasing n_{noise} and n_{blank} in the sequence.

Active T-Maze [29]. The agent is placed at the start position $x = 1$ in a corridor of length L with the oracle at $x = 0$ and the T-junction at $x = L + 1$. The goal $\mathtt{g} \in \{+1, -1\}$ is observable only at the oracle position (cue observed at $x = 0 \to (0, \mathtt{g})$); elsewhere along the corridor the observation is the ambiguous $(0, 0)$. At the T-junction the observation is $(1, 0)$ and the agent must choose to turn up or down based on the remembered cue. The action is the 2D displacement $a_t \in \{(1,0), (-1,0), (0,1), (0,-1)\}$ (`right/left/up/down`).



where s is the start, J the T-junction, $\{1+, 1-\}$ the two goal arms, and the three dots between s and J are a schematic ellipsis denoting a variable-length corridor of L ambiguous $(0, 0)$ cells; the agent must traverse $s \to \mathsf{g} \to s \to \cdots \to \mathsf{J} \to 1\pm$ correctly to succeed. Example with $L = 4$ and $\mathsf{g} = +1$ is

t	0	1	2	3	4	5	6
o_t	$(0, 0)$	$(0, +1)$	$(0, 0)$	$(0, 0)$	$(0, 0)$	$(0, 0)$	$(1, 0)$
a_t	$(-1, 0)$	$(1, 0)$	$(1, 0)$	$(1, 0)$	$(1, 0)$	$(1, 0)$	$(0, +1)$

The two keyframes $\mathcal{X}_{\text{key}} = \{o_1, o_6\}$ are the cue exposure at $t = 1$ ( ) and the T-junction observation at $t = 6$ ( ); the junction is itself a keyframe because the policy must use the $(1, 0)$ token to know when to turn into a goal arm. At the junction the agent must emit $(0, +1)$ if $\mathsf{g} = +1$ and $(0, -1)$ if $\mathsf{g} = -1$. The ambiguous $(0, 0)$ corridor observations at all other timesteps ( ) are background frames. Evaluation is done by stretching the gap of $(0, 0)$ corridor observations between the cue g and the junction J.

C.2 Grid environments

Both grid benchmarks share the MiniGrid [30] interface.

`MiniGrid-MemoryS17Random-v0`

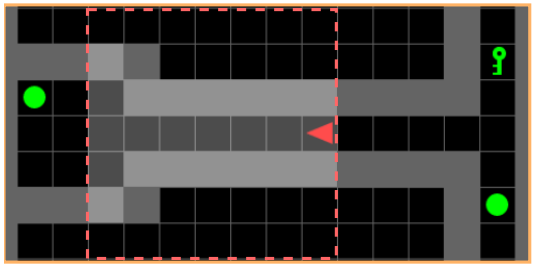


Figure 8: **MiniGrid-Memory environment.** The agent (red triangle) observes only a local (7×7) view of the grid environment (dotted red box). It must navigate to the left room to observe the cue (green ball), then make a U-turn and traverse the corridor to the junction, where it must decide whether to move up or down. To succeed, the agent must move toward the same object observed in the left room.

`LTMB-Hallway-v0`

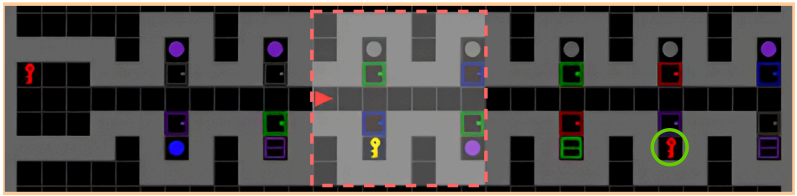


Figure 9: **LTMB-Hallway environment.** The agent (red triangle) observes only a local (7×7) view of the grid environment (dotted red box). It must remember the cue in the left room and then traverse the hallway while matching the remembered cue to the objects behind the locked doors. To succeed, it must open and enter the door containing the same object as the one in the left room. In the example above, the correct target is highlighted with a green circle.

MiniGrid-Memory [30]. A green `Key` or `Ball` is placed in a start room attached to a long corridor; the corridor ends in a T-junction with one `Key` arm and one `Ball` arm (order randomised) (Figure 8). The agent spawns at a random cell along the corridor and must reach the arm matching the start-room object; stepping into the wrong arm terminates with task failure. The keyframe is the start-room object

observation (cue); all corridor observations are background frames. Evaluation is done stretching the corridor length.

LTMB-Hallway [31]. A colored target object (`Key`, `Ball`, or `Box` in one of six colors) is placed in the start room (Figure 9). A horizontal corridor leads past several branching vertical hallways, each containing random objects behind colored doors. Exactly one branch object matches the target in both shape and color, and the agent must open and enter the corresponding hallway door to succeed. Entering any other door results in task failure. The keyframe is the start-room target observation (cue), which must be remembered and matched with hallway targets, while the other corridor observations are background frames. Evaluation is performed by increasing the number of distractor hallways.

C.3 Training dataset

All training datasets contain expert demonstrations only, with no failure trajectories. This matches the requirements of Keyframe Mnemonics, which needs an expert dataset for training. For each baseline we trained two variants per benchmark, differing only in the training-distribution scale: a *small* variant matching the original demonstration distribution of $\mathcal{D}$ used for training Keyframe Mnemonics, and a *large* variant in which the trajectory horizon and the number of variations are scaled up. Training sequence lengths and number of demonstrations ($|\mathcal{D}|$, N in Lemma 1) used in both variants are summarized in Table 2. Policies trained on both variants are evaluated on the same out-of-distribution test distribution, and the variant that achieves the higher test success rate (or lower test MSE for Add) is considered for benchmarking. Some baselines benefit from the larger distribution and from more diverse training data (e.g. DT and TrXL on Scattered-Copy), while others struggle on the *large* variant and generalize better when trained on the *small* dataset (e.g. LSTM-BPTT on T-Maze). See Appendix C.7 for more details.

As discussed in Appendix B.2, we run multiple offline passes over the dataset $\mathcal{D}$ during masked-sampling training of the proxy policy. The number of passes is denoted by $\mathrm{pass}_{\mathrm{ms}}$ in Table 2. We determine this value by starting from a small constant and increasing it by orders of 10 until the requirement on M in Theorem 1 is satisfied. The x-axis multipliers in Figure 2 scale $\mathrm{pass}_{\mathrm{ms}}$; for example, $M = 0.1\times$ for the Add domain means that $\mathrm{pass}_{\mathrm{ms}} = 0.1 \times 300 = 30$ passes were used to train the proxy policy.

Table 2: Training and test sequence lengths and number of demonstrations $|\mathcal{D}|$ per dataset variant. MiniGrid-Memory and LTMB-Hallway use a single dataset (no *large* variant). In the "Seq len" columns, "a–b" denotes a range (lower–upper bound) and "$\sim n$" denotes the maximum sequence length. The last column $\mathrm{pass}_{\mathrm{ms}}$ reports the number of offline rollout passes used to create the masked-sampling cache for proxy training (Appendix B.2).

	Train (*small*)*		**Train** (*large*)		**Test**	
Benchmark	Seq len	$\lvert\mathcal{D}\rvert$	Seq len	$\lvert\mathcal{D}\rvert$	Seq len	$\mathrm{pass}_{\mathrm{ms}}$
Add	5–10	1000	20–30	5000	50–100	300
Scattered-Copy	4	1000	40*	5000	220	30
T-Maze	5–10	100	55–105	5000	$\sim$1005	300
MiniGrid-Memory	$\sim$21	2000	—	—	$\sim$2012	300
LTMB-Hallway	$\sim$46	50,000	—	—	$\sim$425	10

* Dataset used for training Keyframe Mnemonics.

C.4 Training Keyframe Mnemonics policies

Keyframe Mnemonics (KM) uses three networks per benchmark (Algorithm 5): a proxy policy f_θ trained by masked-sampling, a selector π_{s} policy trained with PPO, and a BC policy π_{BC}. Per-benchmark architecture and PPO hyperparameters are summarised in Tables 3 and 4.

In Table 3, note that our formulation allows flexibility in the memory budget k used for proxy and BC policy training, since the inference buffer size can be adjusted after selector training to store a different number of keyframes than what was used during proxy training. The memory budget for the proxy and BC policies is selected empirically for each benchmark, guided by the expected number of task-relevant keyframes. We try a small set of candidate values around this estimate and choose the

Table 3: KM module architectures per benchmark. k: memory budget; d_{feat}: selector feature dimension; d_{hid}: proxy / BC hidden width; H: action-chunk horizon; L: proxy / BC hidden layers. Cells with two values a/b are proxy / BC policy. Architecture of proxy and BC policy can be different per-domain basis.

Benchmark	k	d_{feat}	Proxy / BC arch	d_{hid}	H	L
Add	3	32	ReLU MLP	256	1	3
Scattered-Copy	2/20	32	ReLU MLP / Transformer	256/64	1	3/4
T-Maze	2	16	ReLU MLP	256	1	3
MiniGrid-Memory	2/1	64	CNN and Transformer	128/64	1	4
LTMB-Hallway	2/1	64	CNN and Transformer	128/64	8/1	4

Table 4: KM selector optimization settings per benchmark. PPO uses Adam with the SB3 [64] defaults except where shown (n_{envs} parallel selector environments, total environment steps$_{\text{ppo}}$, rollout length n_{steps}, batch size, n_{epochs} per update, learning rate η_π, and reward normalisation constant λ from Equation 5). The value of λ depends on the type of loss function used for reward objective. The proxy is trained with AdamW (learning rate η_f, batch size).

	Selector PPO							**Proxy**	
Benchmark	n_{envs}	steps$_{\text{ppo}}$	n_{steps}	batch	n_{epochs}	η_π	λ	η_f	batch
Add	4	2×10^5	2048	64	10	3×10^{-4}	10^{-6}	10^{-3}	256
Scattered-Copy	4	2×10^5	2048	64	10	3×10^{-4}	10^{-3}	10^{-3}	256
T-Maze	16	2×10^5	512	64	10	3×10^{-4}	10^{-6}	10^{-3}	256
MiniGrid-Memory	32	3×10^5	512	128	5	3×10^{-4}	10^{-2}	10^{-3}	512
LTMB-Hallway	32	2×10^5	1024	256	5	3×10^{-4}	10^{-2}	10^{-3}	512

smallest k that yields stable proxy policy convergence and produces qualitatively valid mnemonics after selector training. See Appendix G for detailed analysis.

Also note that in Table 3, the proxy policy uses different network architectures across benchmarks. We find that Theorem 1 holds generally, irrespective of network architecture; for instance, the synthetic domains use MLPs whereas grid-world models use CNNs and Transformers. Our choice of ReLU MLPs in synthetic domains is motivated by prior empirical evidence showing that such models exhibit a stronger simplicity bias [65].

C.5 Baselines and benchmarking details

The five benchmarks (Add, Scattered-Copy, T-Maze, MiniGrid-Memory, LTMB-Hallway) share a common trainer pipeline; only the per-baseline hyperparameters and per-task input/output dimensions differ. All baselines are trained with AdamW, a linear warm-up cosine learning-rate schedule (10% of total steps warmup, decaying to 1% of the peak rate), and gradient clipping. Our baseline design and implementation choices are derived from the open-source code of RATE [48]. Additionally, we tuned the training recipe to obtain the best possible baseline results on our benchmarking domains.

Table 5: Trainable parameter counts per benchmark. KM totals sum the proxy, selector and BC policy modules. The baseline architectural choices that determine these counts (layer widths, head counts, embedding dimensions) are inherited from the RATE [48] open-source implementation. Units: K = 10^3, M = 10^6.

Benchmark	LSTM	DLSTM	DT	DMamba	TrXL	ELMUR	**KM (Ours)**
Add	100K	740K	182K	394K	182K	4.4M	**268**K
Scattered-Copy	102K	742K	184K	396K	184K	4.4M	**366**K
T-Maze	100K	740K	246K	395K	246K	4.4M	**266**K
MiniGrid-Memory	641K	1.9M	2.0M	1.5M	997K	4.7M	**1.44**M
LTMB-Hallway	641K	1.9M	2.0M	1.5M	997K	4.7M	**1.45**M

Table 6: Decomposition of the Keyframe Mnemonics parameter budget into the proxy f_θ, the selector policy π_{s}, and the BC policy π_{BC}.

Benchmark	Proxy f_θ	Selector π_{s}	BC policy π_{BC}	**Total**
Add	134K	128	134K	**268**K
Scattered-Copy	157K	2.7K	206K	**366**K
T-Maze	133K	48	133K	**266**K
MiniGrid-Memory	957K	242K	245K	**1.44**M
LTMB-Hallway	959K	242K	245K	**1.45**M

C.6 Parameter-budget comparison

Table 5 compares the total trainable parameter count of every method per benchmark. For Keyframe Mnemonics we report the sum of the proxy, selector, and BC policy parameters. Despite using three networks, the KM total remains well below most of the baselines demonstrating that the architectural decomposition does not come at a parameter cost. Table 6 breaks the KM total down by module. The selector policy is compact (under 1% of the total on the synthetic tasks and roughly 15% on the grid tasks), since it only encodes a single observation at a time; most of the budget sits in the proxy and, the BC policy.

C.7 Benchmark result analysis

We report per-benchmark in-distribution and out-of-distribution performance for every method in Table 7. For each baseline we report the better of policy performance trained on *small* / *large* dataset variants (Appendix C.3); Keyframe Mnemonics is trained only on the *small* variant (except Scattered-Copy that uses *large*[6]). The selected variant per (benchmark, baseline) pair is as follows. On Add, the *small* variant is used for LSTM(BPTT), DLSTM, and the *large* variant for LSTM(TBPTT), DT, DMamba, TrXL, and ELMUR. On Scattered-Copy, the *small* variant is used for LSTM(BPTT) and LSTM(TBPTT), and the *large* variant for all remaining baselines. On T-Maze, the *large* variant is used for LSTM(TBPTT), DT, and TrXL, and the *small* variant for all remaining baselines. The grid benchmarks (MiniGrid-Memory and LTMB-Hallway) use a single dataset.

Table 7: Aggregated benchmark results across all five tasks. Cells show mean $\pm$ std across 5 seeds for Add, Scattered-Copy, and T-Maze, and 3 seeds for MiniGrid-Memory and LTMB-Hallway. For Add, the metric is final-step MSE ($\downarrow$ better); for all others, it is success rate ($\uparrow$ better). "—" marks the LSTM (BPTT) variant on the grid tasks, which we did not train because it required significantly more compute than the other baselines; only TBPTT was trained. See Table 2 for train and test sequence lengths and the number of demonstrations $|\mathcal{D}|$ used for training.

	LSTM(B)	LSTM(T)	DLSTM	DT	DMamba	TrXL	ELMUR	**KM (Ours)**
Add								
Train MSE $\downarrow$	$\mathbf{4 \cdot 10^{-6} \pm 1 \cdot 10^{-6}}$	0.006 ± 0.003	0.167 ± 0.106	$\mathbf{8.1 \cdot 10^{-4} \pm 6.8 \cdot 10^{-4}}$	$\mathbf{2.0 \cdot 10^{-4} \pm 1.3 \cdot 10^{-4}}$	$\mathbf{2.2 \cdot 10^{-4} \pm 1.2 \cdot 10^{-4}}$	0.114 ± 0.014	$\mathbf{3.4 \cdot 10^{-5} \pm 3.6 \cdot 10^{-5}}$
Test MSE $\downarrow$	0.008 ± 0.004	0.040 ± 0.024	0.368 ± 0.186	0.148 ± 0.083	0.160 ± 0.086	0.133 ± 0.064	0.157 ± 0.047	$\mathbf{3.2 \cdot 10^{-5} \pm 3.3 \cdot 10^{-5}}$
Scattered-Copy								
Train Copy SR $\uparrow$	$\mathbf{1.000 \pm 0.000}$	0.100 ± 0.035	0.000 ± 0.000	0.538 ± 0.303	0.000 ± 0.000	0.018 ± 0.016	0.000 ± 0.000	$\mathbf{1.000 \pm 0.000}$
Test Copy SR $\uparrow$	0.160 ± 0.021	0.106 ± 0.030	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	0.000 ± 0.000	$\mathbf{1.000 \pm 0.000}$
T-Maze								
Train SR $\uparrow$	$\mathbf{1.000 \pm 0.000}$	0.330 ± 0.032	0.530 ± 0.055	$\mathbf{1.000 \pm 0.000}$	0.550 ± 0.032	$\mathbf{1.000 \pm 0.000}$	0.682 ± 0.156	$\mathbf{1.000 \pm 0.000}$
Test SR $\uparrow$	0.828 ± 0.236	0.314 ± 0.078	0.478 ± 0.058	0.518 ± 0.043	0.450 ± 0.029	0.500 ± 0.047	0.018 ± 0.035	$\mathbf{1.000 \pm 0.000}$
MiniGrid-Memory								
Train SR $\uparrow$	—	0.823 ± 0.214	0.747 ± 0.067	0.553 ± 0.050	0.203 ± 0.006	0.483 ± 0.035	0.653 ± 0.025	$\mathbf{0.993 \pm 0.012}$
Test SR $\uparrow$	—	0.497 ± 0.072	0.530 ± 0.069	0.547 ± 0.025	0.030 ± 0.000	0.487 ± 0.031	0.300 ± 0.100	$\mathbf{0.997 \pm 0.006}$
LTMB-Hallway								
Train SR $\uparrow$	—	0.037 ± 0.064	0.077 ± 0.031	0.217 ± 0.112	0.000 ± 0.000	0.123 ± 0.015	0.070 ± 0.000	$\mathbf{1.000 \pm 0.000}$
Test SR $\uparrow$	—	0.003 ± 0.006	0.003 ± 0.006	0.013 ± 0.006	0.000 ± 0.000	0.010 ± 0.000	0.010 ± 0.000	$\mathbf{0.993 \pm 0.006}$

Random baseline omitted; its test values are 0.499 ± 0.061 test MSE on Add, 0.000 ± 0.000 test copy SR on Scattered-Copy, 0.000 ± 0.000 test SR on T-Maze, 0.030 ± 0.010 test SR on MiniGrid-Memory, and 0.000 ± 0.000 test SR on LTMB-Hallway.

Memory blur issue. In T-Maze, sequence models such as LSTM (BPTT) and TrXL fit the training distribution well and achieve a train SR of 1.0, but collapse to near-chance performance on longer

[6]For Scattered-Copy, the proxy policy (Stage 1) and selector policy (Stage 2) are trained on the *small* variant, while the BC policy (Stage 3) is trained on the *large* variant.

test horizons. We observe a similar trend in the Add domain. Recurrent memory baselines struggle in these domains because their hidden states or external memories are repeatedly updated over long horizons. Keyframe Mnemonics avoids these failure modes by retaining only keyframes which are relevant for the task which remain unaffected to change in horizon length.

A more severe case is Scattered-Copy, which requires remembering multiple numbers and emitting them in the order they were observed. This task is particularly difficult for baselines that repeatedly update hidden states or external memories, as such updates blur information over time and fail to generalize to longer OOD sequences. In contrast, the KM selector learns to identify relevant keyframes during Stage 2 of training and stores only these keyframes in the buffer, making it easier for the BC policy to learn the copy operation over buffer elements while ignoring noise tokens in the sequence. This highlights a key advantage of our method; since the buffer stores individual tokens, it avoids information blur.

Non-memory baseline. Among the baselines, Decision Transformer (DT) and DMamba do not include an explicit memory mechanism. We include them for completeness in our benchmark evaluation. The observed results are expected, as these models are limited by the context window length used during training; once the sequence length exceeds this window, relevant information falls out of context and the models can no longer predict the correct action. Consequently, they perform well on the training distribution but degrade to near-chance performance on the test distribution.

C.8 Hardware and wall-clock

All training runs were executed on a single NVIDIA RTX 3090 (24 GB) GPU. The three-stage Keyframe Mnemonics pipeline (proxy training, selector PPO, BC policy training) completes across all five benchmarks in ~ 3 h per seed, with the grid benchmarks accounting for the majority of the wall-clock time. The cumulative training time for all baselines on a single synthetic benchmark is under ~ 30 minutes per seed. The cumulative baseline training time is ~ 4 h per seed on MiniGrid-Memory and ~ 6 h per seed on LTMB-Hallway.

D Robot Manipulation Domain Details

This section provides details on the hyperparameters and architectural choices used to produce the `MIKASA-Robo` results in Section 3. For the `Robomimic` domain experiments (Figure 6b), we use the same configuration as `MIKASA-Robo`. Values not stated here take the standard defaults of the underlying libraries.

D.1 `MIKASA-Robo` environments

A set of six representative tasks from the benchmark is shown in Figure 10.

D.2 Pipeline overview

Each `MIKASA-Robo` task is trained in three stages (Algorithm 5), following Sections 2.1–2.3 of the main paper. **(i)** The *proxy policy* f_θ is trained under masked-sampling of past observations, after which its frozen prediction loss serves as the keyframe-discovery objective $J_{\theta^*}(\mathcal{B})$. **(ii)** The *selector policy* π_{s} is trained with PPO against the reward derived from J_{θ^*}, learning to assign high priority to keyframes. **(iii)** The *BC policy* $\pi_{\mathrm{BC}}(a_t \mid o_t, \mathcal{B}_t)$ is trained on the augmented dataset $\mathcal{D}^\dagger$, in which every observation o_t is paired with the priority queue $\mathcal{B}_t$ produced by π_{s}. All three stages use only the offline expert demonstrations $\mathcal{D}$; the simulator is never queried during proxy or selector training. The memory budget is fixed at $k = 3/2$ (proxy / BC policy) throughout, so that π_{BC} sees the current observation alongside two mnemonics drawn from the queue.

D.3 Architectures and parameter counts

All modules share a RESNET-18 visual encoder, initialised from IMAGENET weights and then trained end-to-end. Camera frames are 128×128 RGB; the proprioceptive state concatenates the TCP pose, joint positions, and joint velocities into a 25-dimensional vector. Both overhead and wrist cameras are used. Table 8 summarises the architecture and parameter count of each module.

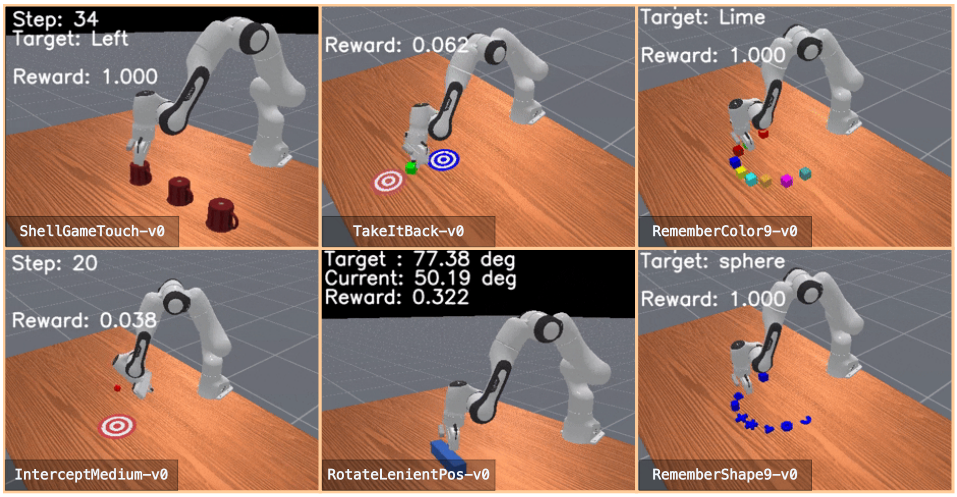


Figure 10: **MIKASA-Robo Environments.** A collage of six representative tasks from the `MIKASA-Robo` memory-intensive robot manipulation benchmark. The benchmark evaluates robot policies across comprehensive memory categories, including object memory, spatial memory, sequential memory, and memory capacity. More details can be found in the reference paper [9].

Table 8: Module architectures, dimensions and parameter counts. d: hidden width, h: attention heads, L: transformer layers, H: action-chunk length. The selector and proxy are training-only; only one BC-policy variant is deployed per task.

Module	**Architecture**	d	h	L	H	**Params**
π_{s} (selector)	ResNet-18 + MLP $[500,\ 500]$	256	—	—	—	15.80 M
f_θ (proxy)	ResNet-18 + Transformer	512	4	4	8	24.13 M
$\pi_{\mathrm{BC}}^{\mathrm{SST}}$	Regression transformer	1024	16	4	1	62.13 M
$\pi_{\mathrm{BC}}^{\mathrm{FM}}$	Flow-matching cross-attn transformer	1024	16	4	8	120.20 M

The single-step regression head ($\pi_{\mathrm{BC}}^{\mathrm{SST}}$) is the default BC-policy. We substitute a flow-matching transformer ($\pi_{\mathrm{BC}}^{\mathrm{FM}}$) only on the four `Intercept*` dynamics tasks (`InterceptMedium` and `InterceptGrab{Slow,Medium,Fast}`), where the higher-frequency control profile benefits from action chunking of length $H = 8$. The flow-matching backbone is a cross-attention transformer in which action tokens self-attend and then cross-attend to visual and proprioceptive context tokens.

D.4 Optimizer and schedule

The proxy and BC policies are trained with AdamW [66]; the selector is trained with PPO [42] using Adam (Table 9). Actions are predicted directly in the simulator's $[-1,\ 1]$ range without output normalization.

Table 9: Optimizer settings for the three training stages.

Stage	**Optimizer**	**LR**	**Weight decay**	**Schedule**
Selector policy π_{s} (PPO)	Adam	5×10^{-5}	0	constant
Proxy policy f_θ	AdamW	1×10^{-4}	0	constant
BC policy $\pi_{\mathrm{BC}}^{\mathrm{SST}}$	AdamW	5×10^{-5}	0.05	constant
BC policy $\pi_{\mathrm{BC}}^{\mathrm{FM}}$	AdamW	5×10^{-5}	0.05	constant

D.5 Stage-1: Proxy policy f_θ

The proxy policy is trained for 60 epochs over the demonstration dataset $\mathcal{D}$ with batch size 128. We use online masked-sampling (Appendix B.2) and at every gradient step we sample a random

binary mask Ψ_k over the past observations h_t with cardinality bounded by k, forming the masked memory buffer $\mathcal{B}_t = \{o_i \in h_t \mid \Psi_k(i) = 1\}$ (Section 2.1). Conditioned on $\mathcal{B}_t$, the proxy is trained to reconstruct the expert action chunk at horizon $H = 8$ under L1 loss.

D.6 Stage-2: Selector policy π_{s}

The selector policy is a PPO actor-critic with the hyperparameters of Table 10. The reward at every step is the normalised log objective defined in Section 2.2, evaluated on the post-update memory buffer $\mathcal{B}_t$. We do not use surprise coefficient for robot manipulation domains. We clip the proxy loss inside the log-reward at $\lambda = 10^{-5}$ to prevent unbounded rewards near the proxy's convergence. The selector observes the same RGB and proprioceptive state input as the proxy policy, and emits a continuous priority score $p_t \in [0, 1]$ per observation; the priority queue uses this score to update $\mathcal{B}_t$ from $\mathcal{B}_{t-1}$ (Algorithm 2). We train the selector for 5×10^6 environment steps across $n_{\mathrm{envs}} = 32$ parallel selector environments.

Table 10: PPO hyperparameters used to train the selector policy π_{s}.

Hyperparameter	Value	Hyperparameter	Value	Hyperparameter	Value
Total environment steps	5×10^6	PPO epochs per update	5	Discount γ	0.99
Parallel envs (n_{envs})	32	Entropy coefficient	0.01	GAE λ_{GAE}	0.95
Rollout length (n_{steps})	1024	Clip range	0.2	Value-function coef. c_{vf}	0.5
Minibatch size	256	Reward floor λ	10^{-5}	Max grad norm	0.5

D.7 Stage-3: BC policy π_{BC}

Optimization budget. The regression head ($\pi_{\mathrm{BC}}^{\mathrm{SST}}$) is trained for 15,000 gradient steps; the flow-matching variant ($\pi_{\mathrm{BC}}^{\mathrm{FM}}$) is trained for 100,000 gradient steps. Batch size is 128 throughout, and the final checkpoint is used for evaluation.

Flow-matching scheduler. We adopt the Euler discrete flow-matching scheduler [67] with 200 training timesteps sampled uniformly at random. At inference we run 5 Euler steps; we found this sufficient on every `Intercept*` task.

Closed-loop control. The BC policy emits an action chunk of length H at every environment step, but only the first action of the chunk is executed before re-querying.

D.8 Camera-image data augmentation

We apply augmentation to all camera images in the buffer during BC policy training. Each of the following operations is applied independently with probability $p = 0.5$: brightness jitter $[0.8, 1.2]$, contrast jitter $[0.8, 1.2]$, saturation jitter $[0.5, 1.5]$, hue jitter $[-0.05, 0.05]$, sharpness adjustment (factor 1.5), and a random affine transform with up to 10° rotation, $\pm 10\%$ translation, and $\pm 10\%$ scaling. The proprioceptive state vector is not augmented. We find this augmentation crucial for training the BC policy to avoid causal confusion [17] induced by the larger context size.

D.9 `MIKASA-Robo` all results

Averaged over the 23 non-zero tasks, Keyframe Mnemonics outperforms Decision Transformer by $+40.4\%$, Conservative Q-Learning by $+42.5\%$, Behavior Cloning by $+37.1\%$, `RATE` by $+30.6\%$, the diffusion-policy baseline by $+36.6\%$, and `ELMUR` by $+13.9\%$ absolute success rate. See Table 11 for complete results.

D.10 Hardware and wall-clock

Training runs were conducted on a mix of NVIDIA A40 (48 GB) and L40S (46 GB) GPU instances. Per-stage wall-clock, averaged across the 23 non-zero `MIKASA-Robo` tasks and rounded, is approximately 1.5 h for the proxy f_θ (60 epochs), $\sim$10 h for the selector π_{s} (5×10^6 environment steps), and $\sim$1.5 h for the BC policy $\pi_{\mathrm{BC}}^{\mathrm{SST}}$ (15,000 gradient steps). End-to-end training therefore takes roughly half a day per SST task ($\sim$13 h). The flow-matching variant $\pi_{\mathrm{BC}}^{\mathrm{FM}}$ trains for 100,000 gradient steps and

Table 11: Success rate (mean±sem over 3 seeds, 100 episodes per seed) on the MIKASA-Robo benchmark. **Bold** marks every method whose mean is within standard error of the best per row.

Environment	RATE	DT	BC	CQL	DP	ELMUR	KM (Ours)
Shell Game Touch	0.92 ± 0.01	0.53 ± 0.07	0.28 ± 0.01	0.16 ± 0.04	0.18 ± 0.02	$\mathbf{0.96 \pm 0.02}$	$\mathbf{0.94 \pm 0.02}$
Shell Game Push	0.78 ± 0.06	0.62 ± 0.14	0.27 ± 0.01	0.25 ± 0.01	0.22 ± 0.03	0.87 ± 0.04	$\mathbf{0.99 \pm 0.01}$
Shell Game Pick	0.02 ± 0.01	0.00 ± 0.00	0.01 ± 0.01	0.00 ± 0.00	0.01 ± 0.00	0.66 ± 0.01	$\mathbf{0.78 \pm 0.08}$
Intercept Slow	0.23 ± 0.02	0.40 ± 0.02	0.37 ± 0.06	0.25 ± 0.01	0.33 ± 0.05	$\mathbf{0.61 \pm 0.04}$	0.48 ± 0.06
Intercept Medium	0.32 ± 0.02	0.56 ± 0.01	0.31 ± 0.14	0.03 ± 0.01	0.68 ± 0.02	0.64 ± 0.03	$\mathbf{0.78 \pm 0.01}$
Intercept Fast	0.30 ± 0.04	0.36 ± 0.04	0.03 ± 0.02	0.02 ± 0.02	0.21 ± 0.05	0.54 ± 0.02	$\mathbf{0.59 \pm 0.04}$
Intercept Grab Slow	0.09 ± 0.03	0.00 ± 0.00	$\mathbf{0.28 \pm 0.18}$	0.03 ± 0.00	0.03 ± 0.01	0.08 ± 0.03	$\mathbf{0.30 \pm 0.04}$
Intercept Grab Medium	$\mathbf{0.09 \pm 0.03}$	0.00 ± 0.00	$\mathbf{0.11 \pm 0.02}$	$\mathbf{0.08 \pm 0.04}$	0.03 ± 0.01	$\mathbf{0.10 \pm 0.04}$	$\mathbf{0.12 \pm 0.08}$
Intercept Grab Fast	0.14 ± 0.03	0.11 ± 0.03	0.09 ± 0.02	0.08 ± 0.03	0.18 ± 0.02	0.20 ± 0.01	$\mathbf{0.37 \pm 0.02}$
Rotate Lenient Pos	0.11 ± 0.04	0.01 ± 0.01	0.15 ± 0.03	0.16 ± 0.02	0.11 ± 0.02	0.24 ± 0.02	$\mathbf{0.52 \pm 0.05}$
Rotate Lenient Pos Neg	0.29 ± 0.03	0.05 ± 0.02	0.22 ± 0.01	0.12 ± 0.02	0.14 ± 0.05	0.26 ± 0.00	$\mathbf{0.71 \pm 0.06}$
Rotate Strict Pos	0.03 ± 0.02	0.05 ± 0.04	0.01 ± 0.00	0.03 ± 0.01	0.06 ± 0.02	0.16 ± 0.00	$\mathbf{0.52 \pm 0.02}$
Rotate Strict Pos Neg	0.08 ± 0.01	0.05 ± 0.03	0.04 ± 0.02	0.04 ± 0.02	0.15 ± 0.01	0.12 ± 0.02	$\mathbf{0.42 \pm 0.07}$
Take It Back	0.42 ± 0.24	0.08 ± 0.04	0.33 ± 0.10	0.04 ± 0.01	0.05 ± 0.02	0.78 ± 0.03	$\mathbf{0.90 \pm 0.04}$
Remember Color3	0.65 ± 0.04	0.01 ± 0.01	0.27 ± 0.03	0.29 ± 0.01	0.32 ± 0.01	0.89 ± 0.07	$\mathbf{0.99 \pm 0.01}$
Remember Color5	0.13 ± 0.03	0.07 ± 0.05	0.12 ± 0.01	0.15 ± 0.02	0.10 ± 0.02	0.19 ± 0.03	$\mathbf{0.46 \pm 0.07}$
Remember Color9	0.09 ± 0.02	0.01 ± 0.01	0.12 ± 0.02	0.15 ± 0.01	0.17 ± 0.01	0.23 ± 0.02	$\mathbf{0.33 \pm 0.03}$
Remember Shape3	0.21 ± 0.04	0.05 ± 0.04	0.31 ± 0.04	0.20 ± 0.10	0.32 ± 0.05	$\mathbf{0.76 \pm 0.01}$	0.63 ± 0.15
Remember Shape5	0.17 ± 0.04	0.04 ± 0.04	0.18 ± 0.01	0.15 ± 0.00	0.21 ± 0.04	0.27 ± 0.02	$\mathbf{0.49 \pm 0.08}$
Remember Shape9	0.05 ± 0.00	0.05 ± 0.02	0.10 ± 0.02	0.14 ± 0.01	0.11 ± 0.02	$\mathbf{0.17 \pm 0.03}$	$\mathbf{0.17 \pm 0.02}$
Remember Shape Color3x2	0.14 ± 0.02	0.04 ± 0.02	0.13 ± 0.02	0.11 ± 0.05	0.14 ± 0.02	0.19 ± 0.02	$\mathbf{0.52 \pm 0.07}$
Remember Shape Color3x3	0.08 ± 0.03	0.06 ± 0.06	0.09 ± 0.02	0.09 ± 0.02	0.16 ± 0.01	0.19 ± 0.02	$\mathbf{0.27 \pm 0.03}$
Remember Shape Color5x3	0.07 ± 0.02	0.01 ± 0.01	$\mathbf{0.09 \pm 0.01}$	$\mathbf{0.09 \pm 0.02}$	$\mathbf{0.11 \pm 0.03}$	$\mathbf{0.13 \pm 0.01}$	$\mathbf{0.14 \pm 0.06}$
Bunch Of Colors3	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Bunch Of Colors5	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Bunch Of Colors7	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Seq Of Colors3	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Seq Of Colors5	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Seq Of Colors7	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Chain Of Colors3	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Chain Of Colors5	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Chain Of Colors7	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00

takes proportionally longer at the BC stage (~ 10 h), bringing the four `Intercept*` flow-matching tasks to roughly a full day each.

E Experimental Details

E.1 Horizon-invariance experiments on synthetic and grid domains

Table 12 reports the consolidated horizon-invariance results underlying Figure 3. We evaluated the trained checkpoints from Appendix C.7, by instantiating the test environment with a target horizon h taken from a pre-defined set per benchmark: Add, Scattered-Copy, T-Maze episode length $h \in \{10, 20, 30, 1000, 10000\}$; Keyframe Mnemonics is additionally evaluated at $h \in \{10^5, 10^6\}$ to test horizon-invariance behavior. MiniGrid-Memory environment max size $h \in \{17, 51, 101, 501, 1001\}$. LTMB-Hallway corridor length $h \in \{8, 50, 100, 500, 1000\}$. Per-cell metrics are mean $\pm$ std across 5 random seeds for the synthetic tasks and 3 random seeds for the grid tasks. For each horizon, every method is evaluated on 5 episodes for the synthetic tasks (Add, Scattered-Copy, T-Maze) and 10 episodes for the grid tasks (MiniGrid-Memory, LTMB-Hallway) per seed.

Within the training horizon range, several baselines achieve strong performance; for example, LSTM(BPTT), DT, and TrXL reach 1.0 SR on T-Maze up to $h = 30$. However, baseline methods collapse once the test horizon exceeds the training horizon by even one order of magnitude. In contrast, Keyframe Mnemonics consistently exhibits horizon-invariant performance. In particular, on synthetic tasks, it generalizes up to five orders of magnitude beyond the training horizon, confirming the horizon-invariance guarantee of Theorem 2 in regimes where the underlying assumptions hold strongly.

E.2 Horizon-invariance experiment on `RememberColor3-`ΔT

The horizon-invariance study (Figure 6a) trains every method at a fixed cue-to-action delay $\Delta T = 15$ and evaluates the trained policies, without retraining, at delays $\Delta T \in$

Table 12: Horizon-invariance results across synthetic and grid benchmarks. Cells report mean ± std of the relevant metric: final-step MSE for Add (↓ better), Copy SR for Scattered-Copy and Episode SR for T-Maze, MiniGrid-Memory and LTMB-Hallway (↑ better). Each benchmark lists the horizons (or environment size for MiniGrid-Memory) evaluated. "—" marks cells for baselines in extreme-horizon experiments as we run it only for Keyframe Mnemonics, or for the LSTM(BPTT) variant on grid tasks, which were trained only with LSTM(TBPTT).

	LSTM(B)	LSTM(T)	DLSTM	DT	DMamba	TrXL	ELMUR	**KM (Ours)**
Add (final-step MSE ↓)								
$h = 10$	$\mathbf{5.0 \cdot 10^{-6} \pm 2.3 \cdot 10^{-6}}$	$3.4 \cdot 10^{-3} \pm 3.5 \cdot 10^{-3}$	0.258±0.216	$\mathbf{6.5 \cdot 10^{-4} \pm 6.2 \cdot 10^{-4}}$	$\mathbf{2.5 \cdot 10^{-4} \pm 1.8 \cdot 10^{-4}}$	$\mathbf{3.9 \cdot 10^{-4} \pm 2.4 \cdot 10^{-4}}$	0.143±0.114	$\mathbf{3.1 \cdot 10^{-5} \pm 3.4 \cdot 10^{-5}}$
$h = 20$	$\mathbf{7.7 \cdot 10^{-5} \pm 6.9 \cdot 10^{-5}}$	$6.8 \cdot 10^{-3} \pm 5.6 \cdot 10^{-3}$	0.330±0.366	$\mathbf{9.2 \cdot 10^{-4} \pm 7.7 \cdot 10^{-4}}$	$\mathbf{2.1 \cdot 10^{-4} \pm 1.9 \cdot 10^{-4}}$	$\mathbf{1.8 \cdot 10^{-4} \pm 1.4 \cdot 10^{-4}}$	0.131±0.090	$\mathbf{2.8 \cdot 10^{-4} \pm 5.7 \cdot 10^{-4}}$
$h = 30$	$\mathbf{3.1 \cdot 10^{-4} \pm 1.9 \cdot 10^{-4}}$	$1.1 \cdot 10^{-2} \pm 8.3 \cdot 10^{-3}$	0.267±0.348	$\mathbf{7.0 \cdot 10^{-4} \pm 7.8 \cdot 10^{-4}}$	$\mathbf{2.2 \cdot 10^{-4} \pm 1.5 \cdot 10^{-4}}$	$\mathbf{1.4 \cdot 10^{-4} \pm 6.1 \cdot 10^{-5}}$	0.141±0.052	$\mathbf{7.7 \cdot 10^{-5} \pm 1.1 \cdot 10^{-4}}$
$h = 10^3$	0.246±0.097	0.127±0.027	0.531±0.455	0.408±0.439	0.387±0.247	0.296±0.216	0.797±0.938	$\mathbf{3.8 \cdot 10^{-5} \pm 4.8 \cdot 10^{-5}}$
$h = 10^4$	0.494±0.320	0.351±0.142	0.218±0.195	0.447±0.398	0.594±0.359	0.634±0.387	0.759±0.465	$\mathbf{3.7 \cdot 10^{-5} \pm 4.0 \cdot 10^{-5}}$
$h = 10^5$	—	—	—	—	—	—	—	$\mathbf{3.8 \cdot 10^{-5} \pm 3.8 \cdot 10^{-5}}$
$h = 10^6$	—	—	—	—	—	—	—	$\mathbf{1.6 \cdot 10^{-5} \pm 3.5 \cdot 10^{-5}}$
Scattered-Copy (Copy SR ↑)								
$h = 10$	0.720±0.110	0.080±0.110	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	**1.000±0.000**
$h = 20$	0.360±0.167	0.080±0.110	0.000±0.000	0.480±0.390	0.000±0.000	0.040±0.089	0.000±0.000	**1.000±0.000**
$h = 30$	0.360±0.167	0.080±0.110	0.000±0.000	0.040±0.089	0.000±0.000	0.000±0.000	0.000±0.000	**1.000±0.000**
$h = 10^3$	0.120±0.179	0.160±0.167	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	**1.000±0.000**
$h = 10^4$	0.000±0.000	0.040±0.089	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	**1.000±0.000**
$h = 10^5$	—	—	—	—	—	—	—	**1.000±0.000**
$h = 10^6$	—	—	—	—	—	—	—	**1.000±0.000**
T-Maze (Episode SR ↑)								
$h = 10$	**1.000±0.000**	0.360±0.219	0.680±0.179	**1.000±0.000**	0.560±0.261	**1.000±0.000**	0.400±0.200	**1.000±0.000**
$h = 20$	**1.000±0.000**	0.320±0.179	0.560±0.329	**1.000±0.000**	0.520±0.335	**1.000±0.000**	0.280±0.228	**1.000±0.000**
$h = 30$	**1.000±0.000**	0.280±0.335	0.640±0.167	**1.000±0.000**	0.520±0.228	**1.000±0.000**	0.520±0.228	**1.000±0.000**
$h = 10^3$	0.800±0.283	0.320±0.228	0.320±0.228	0.320±0.228	0.360±0.261	0.520±0.303	0.000±0.000	**1.000±0.000**
$h = 10^4$	0.840±0.261	0.320±0.110	0.320±0.110	0.320±0.110	0.360±0.167	0.440±0.219	0.000±0.000	**1.000±0.000**
$h = 10^5$	—	—	—	—	—	—	—	**1.000±0.000**
$h = 10^6$	—	—	—	—	—	—	—	**1.000±0.000**
MiniGrid-Memory (Episode SR ↑)								
$h = 17$	—	0.767±0.404	0.800±0.000	0.333±0.153	0.500±0.000	0.600±0.000	0.467±0.058	**1.000±0.000**
$h = 51$	—	0.667±0.321	0.533±0.231	0.533±0.058	0.300±0.000	0.600±0.000	0.367±0.115	**1.000±0.000**
$h = 101$	—	0.600±0.265	0.500±0.173	0.333±0.153	0.100±0.000	0.500±0.100	0.333±0.153	**1.000±0.000**
$h = 501$	—	0.500±0.173	0.500±0.173	0.433±0.231	0.000±0.000	0.300±0.100	0.333±0.153	**1.000±0.000**
$h = 1001$	—	0.467±0.208	0.500±0.173	0.433±0.153	0.000±0.000	0.367±0.115	0.300±0.173	**1.000±0.000**
LTMB-Hallway (Episode SR ↑)								
$h = 8$	—	0.033±0.058	0.033±0.058	0.133±0.058	0.000±0.000	0.100±0.000	0.100±0.000	**1.000±0.000**
$h = 50$	—	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	**1.000±0.000**
$h = 100$	—	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	**0.933±0.058**
$h = 500$	—	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	**0.967±0.058**
$h = 1000$	—	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	0.000±0.000	**0.933±0.058**

Random baseline omitted; its test SR remains low across every (benchmark, horizon) cell, consistent with performance shown in Table 7.

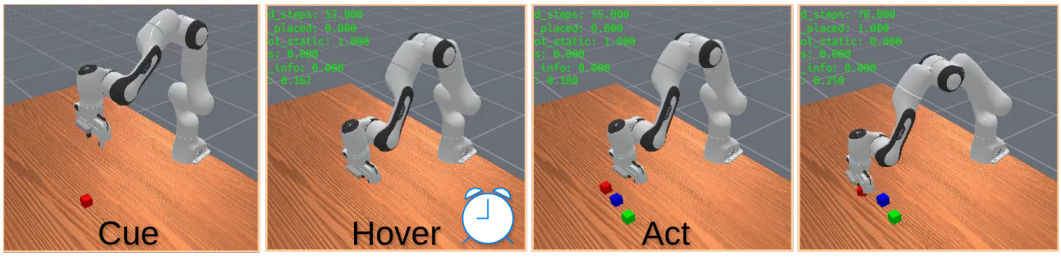


Figure 11: Hover time is varied across evaluations resulting in catastrophic failure for the baselines, while KM maintains it's performance even at 250 timesteps of hover. Training data had 15 timesteps of hover.

$\{15, 35, 55, 75, 95, 115, 135, 150, 200, 250\}$. The training set is built from the released `RememberColor3-v0` demonstrations, whose default cue-to-action delay is $\Delta T_{\text{orig}} = 5$.

Hover-augmented data. We re-replay each successful expert demonstration in a live `ManiSkill` simulator with the cue-to-action delay extended to $\Delta T = 15$ and a 10-step *extra hover* phase spliced into the original action tape immediately after the cue disappears (Figure 11). During the hover phase the gripper is held and small net-zero noise is added to the arm for the entire delay time.

Training on the hover dataset. We run the standard three-stage KM pipeline of Appendix D.2 on the hover trajectories. Every hyperparameter matches the corresponding entries for `RememberColor3` in Tables 8 ($k = 3$, proxy chunk $H = 8$, SST policy with $H = 1$, 15,000 BC gradient steps).

We train `PTP` using their official implementation on our dataset with prediction horizon of 16, observation length of 8 and a sub-sampling rate of 8 to roughly cover the entire episode of length

70. This indicates why we see steeper deterioration in the performance of PTP on and after 75 steps of time delay. We train ELMUR on the same trajectories using its published RememberColor3 recipe, with the context length extended to cover the full 70-step hover episode. All other hyperparameters (architecture, optimizer, schedule, batch size, number of epochs) are left at the authors' defaults.

Horizon-invariance evaluation. The trained policy is evaluated at every target delay ΔT by re-instantiating the simulator with the requested ΔT; no retraining or fine-tuning is performed at any $\Delta T \neq 15$. Following the MIKASA-Robo convention, an episode is counted as a success only if the binary task-completion reward is reached before the time budget elapses. We report mean $\pm$ standard error across three independent environment seeds, 100 episodes per seed. Results are plotted in Figure 6a and details can be found in Table 13.

Results in Table 13 show that KM preserves horizon-invariant performance in memory-intensive robot manipulation with high-dimensional visual data. ELMUR matches the other methods at the training delay ($\Delta T = 15$), but its external memory is overwritten as the delay increases, causing success to collapse to zero for every $\Delta T \geq 35$. PTP retains its training-time performance up to roughly $\Delta T = 55$ and then degrades monotonically, dropping below 10% by $\Delta T = 200$. In contrast, Keyframe Mnemonics remains nearly flat across the longer horizons, retaining ≈ 0.65 success rate at $\Delta T = 250$, i.e., $\sim 17\times$ the training delay, without any architectural change or fine-tuning.

Table 13: RememberColor3-ΔT success rate under cue-to-action delays. Training delay is $\Delta T = 15$; all larger values are out-of-distribution. Mean $\pm$ sem over 3 seeds, 100 episodes per seed.

ΔT	PTP [49]	ELMUR [16]	**KM (Ours)**
15	0.66 ± 0.03	0.61 ± 0.07	$\mathbf{0.72 \pm 0.07}$
35	0.72 ± 0.04	0.00 ± 0.00	$\mathbf{0.69 \pm 0.11}$
55	$\mathbf{0.73 \pm 0.05}$	0.00 ± 0.00	0.69 ± 0.11
75	0.55 ± 0.07	0.00 ± 0.00	$\mathbf{0.69 \pm 0.08}$
95	0.43 ± 0.06	0.00 ± 0.00	$\mathbf{0.65 \pm 0.08}$
115	0.31 ± 0.05	0.00 ± 0.00	$\mathbf{0.66 \pm 0.09}$
135	0.23 ± 0.03	0.00 ± 0.00	$\mathbf{0.66 \pm 0.09}$
150	0.17 ± 0.03	0.00 ± 0.00	$\mathbf{0.65 \pm 0.09}$
200	0.07 ± 0.02	0.00 ± 0.00	$\mathbf{0.64 \pm 0.08}$
250	0.07 ± 0.03	0.00 ± 0.00	$\mathbf{0.65 \pm 0.09}$

E.3 Reproducibility

Released code, per-task configurations, and trained checkpoints, together with launch scripts, are available at https://keyframe-mnemonics.github.io.

F Ablation Study

We report ablation results over 5 seeds for the synthetic domains and 3 seeds for the grid and robot domains unless noted, with 100 evaluation episodes per seed. All domains are evaluated at out-of-distribution horizons.

F.1 Learned selector against a random selector

We replace the learned selector with a random selector π_{rnd} that assigns priorities $p \sim \mathcal{U}(0, 1)$ and train the BC policy using Stage 3 (Algorithm 5). The resulting policy performs worse at every delay interval, demonstrating that the learned selector is critical for the memory task.

Table 14: Learned selector against the random selector π_{rnd} on MIKASA-Robo RememberColor3-ΔT. Reported mean$\pm$sem success rate (%) $\uparrow$.

ΔT	Learned selector	Random selector (π_{rnd})
15	$\mathbf{72.3 \pm 5.8}$	31.3 ± 3.1
35	$\mathbf{69.0 \pm 9.2}$	28.7 ± 2.9
55	$\mathbf{69.0 \pm 8.8}$	28.3 ± 2.1
75	$\mathbf{68.7 \pm 6.6}$	35.3 ± 1.2
95	$\mathbf{65.0 \pm 6.4}$	28.7 ± 4.0
115	$\mathbf{66.3 \pm 7.6}$	28.3 ± 4.1
135	$\mathbf{65.7 \pm 7.4}$	33.0 ± 1.4
150	$\mathbf{65.3 \pm 7.6}$	29.0 ± 0.8
200	$\mathbf{64.3 \pm 6.9}$	31.7 ± 1.7
250	$\mathbf{65.3 \pm 7.6}$	26.7 ± 1.2

F.2 Selector discount factor

We vary the discount factor γ of the PPO selector to optimize over a frozen proxy policy. At $\gamma = 0$ the selector greedily optimizes the per-step proxy loss. For T-Maze we report the active (default) and passive variations [68]. Greedy optimization fails at $\gamma = 0$ on every domain except active T-Maze, and performance recovers as γ increases. In active T-Maze, the

action changes right after the keyframe is observed, so the immediate change in proxy loss is sufficient. In the other domains, the gap between the keyframe and the decision-critical action spans multiple steps, and a higher γ is necessary. Keyframe identification from the proxy loss is therefore a delayed credit-assignment problem, which explains our choice of using RL for selector training (Section 2.2).

Table 15: Effect of the PPO discount factor γ. Reported mean±sd.

Domain	Metric	$\gamma = 0$	$\gamma = 0.5$	$\gamma = 0.9$	$\gamma = 0.99$ (default)
T-Maze (active)	Episode SR ↑	**100.0 ± 0.0**	**100.0 ± 0.0**	**100.0 ± 0.0**	**100.0 ± 0.0**
T-Maze (passive)	Episode SR ↑	48.8 ± 6.6	**100.0 ± 0.0**	**100.0 ± 0.0**	**100.0 ± 0.0**
Add	Final MSE ↓	1.84e-01 ± 1.77e-03	1.25e-02 ± 6.86e-03	**1.28e-05 ± 5.67e-06**	**1.28e-05 ± 5.70e-06**
Scattered-Copy	Copy SR ↑	0.0 ± 0.0	**100.0 ± 0.0**	**100.0 ± 0.0**	**100.0 ± 0.0**

F.3 Surprise coefficient

We change the temperature κ of the surprise coefficient in Equation 5. Off in Table 16 corresponds to an unscaled reward, which is the $\kappa \to \infty$ limit where surprise coefficient equals 1. For every task we first train with the unscaled reward and add surprise scaling only if it improves performance. $\kappa = 0.2$ has the widest usable range and is our default for the tasks that need scaling (Add, Scattered-Copy and MiniGrid-Memory).

Table 16: Effect of the surprise coefficient across temperatures. Bold marks the best setting per domain. Reported mean±sd.

Domain	Metric	$\kappa = 0.02$	$\kappa = 0.2$	$\kappa = 2$	off
T-Maze	Episode SR ↑	0.0 ± 0.0	**100.0 ± 0.0**	**100.0 ± 0.0**	**100.0 ± 0.0**
Add	Final MSE ↓	**1.28e-05 ± 5.7e-06**	**1.28e-05 ± 5.7e-06**	1.84e-01 ± 1.8e-03	1.84e-01 ± 1.8e-03
Scattered-Copy	Copy SR ↑	**100.0 ± 0.0**	**100.0 ± 0.0**	**100.0 ± 0.0**	0.0 ± 0.0
MiniGrid-Memory	Success rate ↑	**100.0 ± 0.0**	**100.0 ± 0.0**	3.0 ± 0.0	3.7 ± 0.6
LTMB-Hallway	Success rate ↑	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	**99.7 ± 0.6**
RememberColor3	Success rate ↑	25.0 ± 3.3	29.3 ± 6.2	34.7 ± 6.9	**99.0 ± 0.0**

F.4 Current observation in proxy training

Including o_t during proxy training collapses MiniGrid-Memory and RememberColor3 (Table 17), as the proxy overfits to the current observation. This is consistent with Proposition 1, which requires that no low-complexity non-invariant shortcut exists in the proxy training data. The current observation is helpful for learning the final policy, but not for learning a reward function for keyframe identification.

Table 17: Effect of including the current observation o_t in the proxy input. Reported mean±sd.

Domain	Metric	Proxy without o_t (default)	Proxy with o_t
T-Maze	Episode SR ↑	100.0 ± 0.0	100.0 ± 0.0
Add	Final MSE ↓	1.28e-05 ± 5.7e-06	2.20e-05 ± 1.3e-05
Scattered-Copy	Copy SR ↑	100.0 ± 0.0	100.0 ± 0.0
MiniGrid-Memory	Success rate ↑	**100.0 ± 0.0**	33.3 ± 26.3
LTMB-Hallway	Success rate ↑	**99.7 ± 0.6**	93.0 ± 9.5
RememberColor3	Success rate ↑	**99.0 ± 0.0**	18.7 ± 5.9

F.5 Queue tie-break rule

The memory buffer uses a FIFO queue (Algorithm 2) during selector training and a LIFO queue (Algorithm 3) at inference. The two differ in which slot is evicted when several observations are tied at the lowest priority. We vary this rule independently at training and inference. On the synthetic and grid domains, all four combinations are identical (Table 18). The rule matters on RememberColor3-ΔT, where the default FIFO → LIFO beats its reverse at 9 of 10 retention intervals (Table 19).

Table 18: Queue tie-break rule, train → inference. Reported mean±sd.

Domain	Metric	FIFO → FIFO	FIFO → LIFO (default)	LIFO → FIFO	LIFO → LIFO
T-Maze	Episode SR ↑	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
Add	Final MSE ↓	1.28e-05 ± 5.7e-06	1.28e-05 ± 5.7e-06	1.28e-05 ± 5.7e-06	1.28e-05 ± 5.7e-06
Scattered-Copy	Copy SR ↑	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
MiniGrid-Memory	Success rate ↑	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0	100.0 ± 0.0
LTMB-Hallway	Success rate ↑	99.6 ± 0.5	99.6 ± 0.5	99.6 ± 0.5	99.6 ± 0.5
RememberColor3	Success rate ↑	93.7 ± 4.5	**99.0 ± 0.0**	98.7 ± 1.3	96.0 ± 2.2

Table 19: Queue tie-break rule on RememberColor3-ΔT. Reported mean±sem success rate ↑.

ΔT	FIFO → LIFO (default)	LIFO → FIFO
15	72.3 ± 5.8	**75.0 ± 1.6**
35	**69.0 ± 9.2**	66.0 ± 3.6
55	**69.0 ± 8.8**	66.0 ± 2.4
75	**68.7 ± 6.6**	59.3 ± 3.7
95	**65.0 ± 6.4**	59.3 ± 1.7
115	**66.3 ± 7.6**	58.3 ± 2.5
135	**65.7 ± 7.4**	60.3 ± 1.7
150	**65.3 ± 7.6**	59.3 ± 2.5
200	**64.3 ± 6.9**	58.0 ± 1.6
250	**65.3 ± 7.6**	57.7 ± 3.8

F.6 Memory buffer size

We scale the memory buffer size k to $0.5\times$ and $2\times$ the default value and retrain all three stages. Halving k degrades performance across all domains, while doubling it helps on some domains and hurts on others. A larger k can make the KM policy harder to train, and our default values serve as a minimum-capacity floor, which were chosen per domain independently.

Table 20: Effect of scaling the memory buffer size k. Reported mean±sd.

Domain	Metric	$0.5\times$	$1\times$ (default)	$2\times$
T-Maze	Episode SR ↑	40.0 ± 23.2	**100.0 ± 0.0**	**100.0 ± 0.0**
Add	Final MSE ↓	9.55e-02 ± 2.38e-02	**1.28e-05 ± 5.7e-06**	2.33e-05 ± 2.03e-05
Scattered-Copy	Copy SR ↑	0.0 ± 0.0	**100.0 ± 0.0**	80.0 ± 44.7
MiniGrid-Memory	Success rate ↑	35.3 ± 26.4	**100.0 ± 0.0**	20.7 ± 27.2
LTMB-Hallway	Success rate ↑	0.0 ± 0.0	81.7 ± 15.0	**99.3 ± 1.2**
RememberColor3	Success rate ↑	94.7 ± 2.5	99.0 ± 0.0	**100.0 ± 0.0**

G Additional Analysis

G.1 Interpretability of discovered keyframes

For a well-trained selector, the *mnemonics* buffer $\mathcal{B}_t$ should accumulate keyframes that are information-critical for the task and provide necessary context for the downstream BC policy to generate actions unambiguously. Figure 4 shows the buffer contents for our visual benchmark domains. We observe that the buffer keyframes qualitatively capture the belief state for each task by retaining observations with task-relevant cues. The discovery process is fully self-supervised, as the selector is trained only from expert demonstration state-action pairs without any additional temporal keyframe annotation. These qualitative results highlight the interpretability of Keyframe Mnemonics: the buffer contains prior observations that can be inspected to understand what the BC policy relies on when taking a particular action in a partially observable environment.

G.2 Selector priority behavior

In Figure 12 we evaluate trained selector policy π_{s} across synthetic domains to qualitatively assess how well the learned priorities align with the oracle keyframes defined in Appendix C.1. We find that

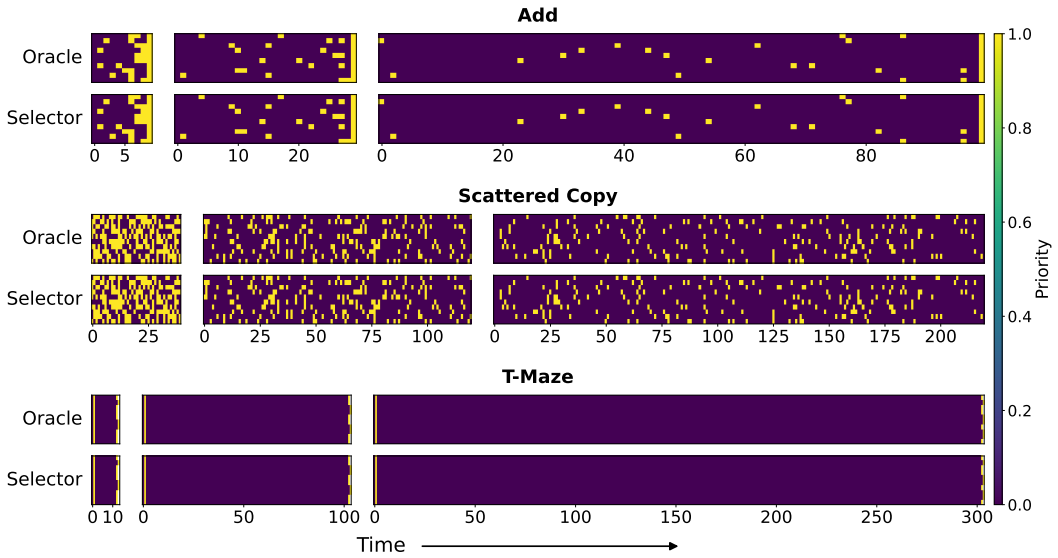


Figure 12: **Trained selector recovers the keyframe oracle as a near-optimal function.** Per-step selector priorities (bottom row of each pair) are compared against the oracle keyframe indicator (top row of each pair) on three synthetic memory benchmarks. Each benchmark is evaluated at three increasing episode horizons, where the longest horizon extends one order of magnitude beyond the training horizon. For each panel, we render 10 evaluation episodes (rows) over time (columns), where each column corresponds to a single environment step and color encodes either the selector output ($p_t \in [0, 1]$) or the binary oracle indicator (1 for keyframes and 0 otherwise). The oracle definition is problem-specific, as discussed in Appendix C.1. Across all cells, the selector priority agrees with the oracle. After squashing the raw selector outputs to $[0, 1]$, the mean selector priority is exactly 1.00 on keyframes and 0.00 on background frames. This empirical result shows that the selector policy π_s learns a near-optimal keyframe selection function for domains where our assumptions hold strongly.

the selector learns a near-optimal selection function that agrees with the oracle, highlighting that the proxy objective provides a strong signal for keyframe discovery, consistent with Corollary 1. During Stage 2 RL training (Algorithm 4), the selector explores priority assignments and learns to commit high priorities to decision-relevant keyframes. This qualitatively explains the horizon-invariant results in Table 12, where BC policies achieve perfect performance over horizons orders of magnitude beyond training. Since the selector is Markovian and depends only on the current observation, its behavior remains invariant to horizon length, allowing it to admit only informative keyframes into the buffer at inference time.

G.3 BC Policy activation map

To understand whether the discovered keyframes are useful to the behavior cloning policy, we use Grad-CAM [69, 70], an explainable AI technique commonly used to visualize which parts of the input a policy focuses on when generating actions. In Figure 13, we visualize activation maps at decision-critical timesteps in grid and robot manipulation domains, where the BC policy must rely on memory-buffer information to generate the correct action. We observe that the BC policy attends to information cues from both the buffer and the current observation to infer the POMDP belief state and act correctly. In the grid domain, the heatmap is concentrated around the matching object cue. In `RememberColor3`, activations concentrate around the blue colored cube in both the keyframe and the current observation. In `TakeItBack`, activations appear around the initial cube location in the first keyframe, while the second keyframe captures temporal task-progress information just before the robot begins pushing the cube, helping disambiguate the state so that the policy pulls the cube back to its initial location. These results show that the keyframes discovered by our method are decision-relevant and provide necessary context for BC policies to disambiguate actions under partial observability, even in high-dimensional robot visual domains.

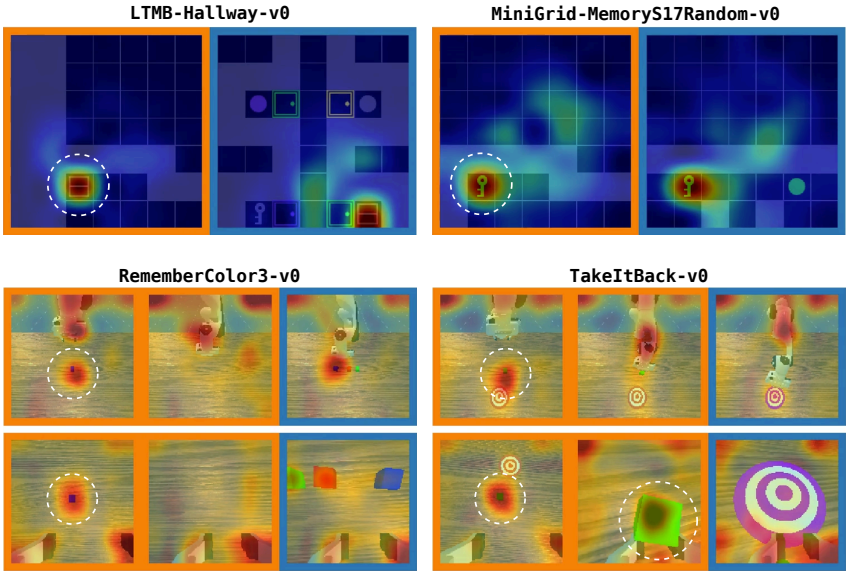


Figure 13: **BC policy activation map on mnemonics.** Grad-CAM [69, 70] activation maps of the behavior cloning policy π_{BC} on buffer images at decision-critical steps, where hot regions indicate inputs driving the predicted action. Frames with ■ boundaries represent discovered keyframes, while the frame with a ■ boundary shows the agent's current observation. Across both grid and high-dimensional visual domains, the BC policy uses task-specific cue information from the *discovered* keyframes to generate actions. This highlights the utility of mnemonics for disambiguating policy actions under partial observability. See `https://keyframe-mnemonics.github.io` for activation map videos over policy rollouts.

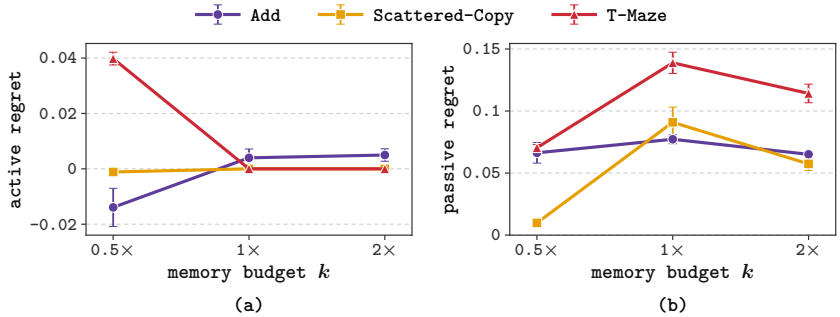


Figure 14: **Memory regret as a function of buffer capacity k.** The $1\times$ setting corresponds to the proxy training memory budget k in Table 3. **(a)** Active regret decreases as buffer capacity becomes sufficient to retain the true keyframes, with domain-specific behavior when k is below the true keyframe count. **(b)** Passive regret remains low under insufficient capacity and decreases again for larger buffers, where random selection is more likely to retain keyframes.

G.4 Regret on low memory budget k

Figure 14 analyzes how memory regret (Section 3.1) changes with buffer capacity k for synthetic domains. In T-Maze, active regret remains high when $k = 1$ ($\lceil 0.5 \times 2 \rceil$), which is below the true number of keyframes (2), because the buffer cannot retain all required keyframes. In Add, active regret becomes negative when $k = 2$ ($\lceil 0.5 \times 3 \rceil$), also below the true keyframe count (3). This occurs because the selector prioritizes the two marked numbers over the emit step, whereas the oracle assigns equal priority to all three keyframes. Under the low memory budget, the oracle evicts the first marked step, leading to higher proxy loss. It shows that the selector can learn a non-uniform priority policy that favors keyframes with higher mutual information with the action. For Scattered-Copy, active regret remains close to zero since the oracle and selector perform similarly across memory

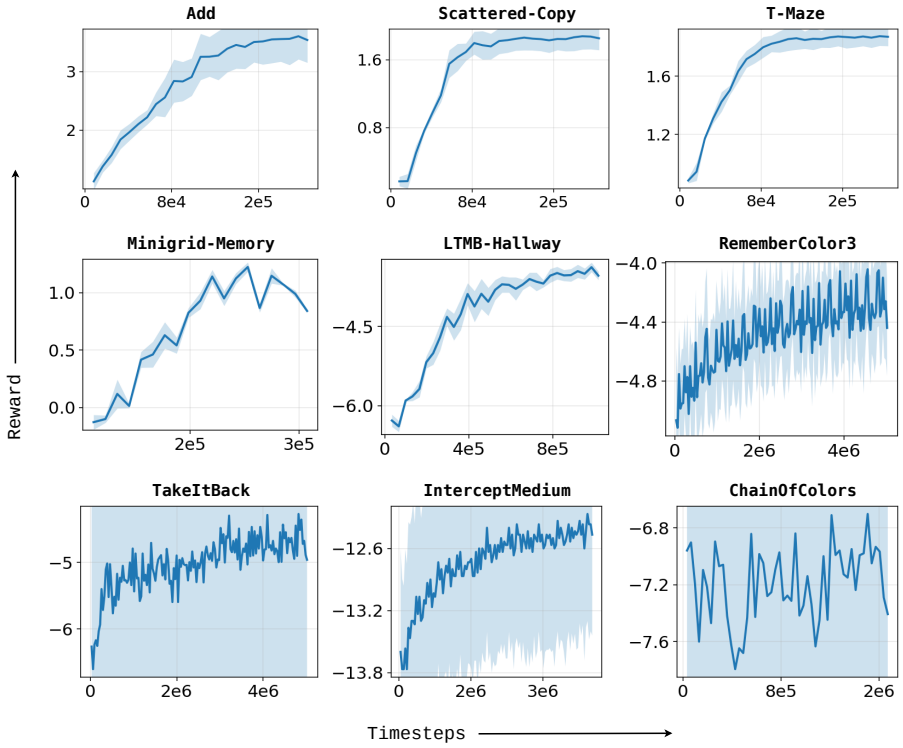


Figure 15: **Selector training PPO learning curves.** The selector policy converges within 200K environment steps for synthetic domains and 5M steps for robot manipulation domains. For tasks where optimization is difficult, e.g., `ChainOfColors`, the learning curve is noisy and shows modest reward improvement, indicating that some of our assumptions may not hold strongly in the environment.

budgets. Passive regret remains low when the buffer size is smaller than the true keyframe count because the proxy cannot converge to an invariant hypothesis under constrained memory. At the $2\times$ buffer size, passive regret is lower than at $1\times$, since a larger buffer increases the probability that a random selector retains keyframes.

G.5 PPO reward plots

Figure 15 shows the learning curves for training the selector policy π_{s} on the frozen proxy objective (Section 2.2) using PPO [42]. For synthetic domains where our assumptions hold strongly, we observe clear reward improvement, with the selector converging within 200K environment (Algorithm 4) steps. For robot manipulation domains, the selector converges within 5M steps. The plot for `ChainOfColors` illustrates a task that is more difficult to optimize, exhibiting a noisy learning curve. We observed that such noise typically appears when the proxy policy does not converge properly, either because it is undertrained or because our assumptions do not hold well in the environment.

G.6 Limitation discussion

Our Assumption 1 is restrictive, as it requires keyframes to be individually identifiable from a single observation because the selector policy $\pi_{\mathrm{s}}(p_t \mid o_t)$ is Markovian. This formulation allows selector training to scale without recurrent modules, thereby avoiding backpropagation through time. Hence, our method is best suited for sparse, frame-localizable memory tasks. However, problems that do not satisfy the assumption may require multiple keyframes to represent the same underlying information, leading to multiple information-equivalent buffer states that achieve similar objective values under the proxy policy. In such cases, it becomes challenging to retain only one mnemonic set among several equivalent alternatives.

Our Assumption 2 requires action ambiguity to be strongly correlated with a sparse set of keyframes. In other words, the action distribution must exhibit sufficient multimodality that can be disambiguated by the remembered keyframes. In tasks where actions are mostly local and exhibit low variance, actions may carry little mutual information with the buffer observations. In such a regime, the mutual information gap in Theorem 1 becomes negligible, and the proxy policy may fail to converge because the invariant hypothesis is difficult to learn from a sparse memory buffer. We observed this failure mode on the VizDoom-Two-Colors [24] task, where the agent observes a color cue and must later choose the corresponding target, but the discrete navigation actions remain largely local.

We found that the proxy policy struggles to converge to the true invariant hypothesis when expert actions at decision-critical timesteps depend simultaneously on multiple unique keyframes. This occurs because the probability that masked-sampling selects all relevant keyframes during proxy training decreases as the number of action-relevant keyframes increases, making it difficult to learn tasks requiring high memory capacity. A similar issue arises in very long demonstrations with few keyframes, where keyframe sparsity further reduces the probability of sampling the relevant frames. A straightforward way to address this issue is to increase the training memory budget, which makes sampling keyframes into the buffer more likely and also increases the support size M, thereby enforcing Theorem 1 more effectively. Another promising direction is to train proxy policies on shorter low-memory-capacity tasks, learn specialized selectors for each, and compose them to support high-memory-capacity BC policies, which we leave for future work.

The PPO training time dominates the computational cost in robot-domain experiments. Our current implementation uses SB3 [64], which introduces nontrivial CPU-GPU overhead and becomes the main bottleneck. The selector training environment (Algorithm 4) primarily requires proxy policy inference, which can in principle be executed entirely on the GPU. In future work, we plan to use a GPU-native PPO implementation to accelerate selector training.

H Extended Related Works

Language as a proxy for state tracking. Recent methods have used segment-level language-annotated demonstrations to finetune vision-language models (VLMs) that generate autoregressive subgoal commands for policy steering [71, 72]. While language provides high-level semantic guidance, it lacks the spatio-temporal detail present in visual observations. In contrast, our approach does not strictly require language annotations for training. Instead, we view Keyframe Mnemonics as complementary to such methods, with informative keyframes addressing partial observability and language supporting abstract reasoning over past events.

Masking-induced invariant feature learning. Prior works have used masking in self-supervised learning over language tokens [39] and spatial image patches [40] to learn invariant features from input data. Our keyframe discovery process instead applies masking over the temporal history of individual observations. We hypothesize that this induces a similar regularization effect: random input sampling increases the difficulty of learning complex, non-invariant features, thereby biasing the model toward invariant hypotheses that better capture true correlations over spurious ones.

Instance-wise feature selection. Prior work learns to select task-relevant input features with a selector–predictor architecture, using a Gumbel-softmax relaxation to sample a fixed number of features [73] or an actor-critic objective against a full-feature baseline [74]. ATK [75] applies a Gumbel-softmax mask with an $L1$ penalty to select task-relevant keypoints for robot policies. These methods select from the current input, assuming it suffices for decision-making, and do not transfer to non-Markovian settings where selection over the full history is intractable. Keyframe Mnemonics instead uses masked sampling over history subsets bounded by the memory budget k.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: We have clearly stated the contributions in the introduction and have ensured that the Methodology, and the Experiments and Results Analysis sections back up the three contributions clearly.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper has a limitations section where we have stated clearly the limitations of the current work.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: In the methodology section and the appendix we have attached mathematical proofs of our claims which are correct in our opinion.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We are submitting self contained code to run our framework on two of the domains demonstrated in our results. We have aimed to provide parameters for our architectures where appropriate. We have not collected any data as part of this paper.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [Yes]

   Justification: We have released two self contained domains with our architecture that are runnable. This should allow the reviewers to verify our claims. We have also attached our webpage with more information and access to github.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: The results and Appendix B aim to provide complete experimental details for reproducibility.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [Yes]

   Justification: The error bars are visible in each of the figures and claims are made with respect to these error bars.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Appendix C and D specified compute times for our methods and computational resources used.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: This work has not collected any human data. We have listed computational costs and released runnable code to demonstrate reproducibility. Our work is in introductory stages so does not have comparable societal impact to existing generative AI technologies as of now. The goal is to improve memory of neural networks to eventually perform long horizon tasks on robots which would be more directly useful to our society.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

Justification: The work on improving memory for neural networks is not practical yet for direct societal interaction. This work is on artificial benchmark and simulated robot tasks which is not meaningful to society at large in its current state.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The work is too preliminary to have a high risk of misuse in a societal scale.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We have cited the papers whose code and/or data we have used to run our experiments. All other code is our own.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.

- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

    Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

    Answer: [Yes]

    Justification: We are releasing code as part of the supplementary material and on our webpage. No human subjects data was collected or used as part of this project.

    Guidelines:

    - The answer [N/A] means that the paper does not release new assets.
    - Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
    - The paper should discuss whether and how consent was obtained from people whose asset is used.
    - At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

    Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

    Answer: [N/A]

    Justification: We did not collect any human subjects data.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
    - According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: We did not collect any human subjects data.

    Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs were not used to develop the core idea or experimental design of this research.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.